GenESyS

Simulation Platform

Practical Guide for GenESyS Users and Developers

Prof. Rafael Luiz Cancian, Dr. Ing.

www.ine.ufsc.br/rafael.cancian

July 2026

Contents

Preface

This book presents the *GenESyS* simulator as a tool for practical use and as a development platform. Its objective is twofold. Firstly, it serves as a manual for the user who wants to install the system, understand how it is used, build models, execute them and interpret their results. Secondly, to serve as a manual for the developer who wants to understand the internal architecture of the simulator and extend it through new components, new *data definitions*, new language functions, new tools and new applications.

Unlike a broad text on systems modeling and simulation, this book is deliberately centered on *software*. General concepts appear only when they are necessary to clarify the functioning of *GenESyS*, its internal organization or the correct way to use some of its resources. Therefore, the focus remains on what really interests the reader of this volume: how the simulator works, how it is used and how it can be improved.

The book is organized with a simple progression. The initial chapters deal with *GenESyS* as a direct use tool, presenting installation, compilation, execution and the most important ways of interacting with the simulator. The text then moves on to the programmatic use of the library and the essential structure of the model and simulation core. Finally, the book focuses on *GenESyS* as an extensible platform, covering core development elements such as new components, *plugins*, parser, tools and applications.

This organization seeks to serve two reader profiles without excessively fragmenting the material. The first is the reader who just wants to use the simulator. The second is the reader who intends to collaborate with the project or adapt it to new needs. Although these profiles have different objectives, they both benefit from a clear understanding of the relationship between external use and internal structure of the system. For this reason, this book was written to allow progressive reading: Those who just want to use the simulator can initially concentrate on the entry and use chapters; Those who wish to develop it can advance to the final chapters, focused on architecture and extension.

It is also important to highlight that this volume focuses on concrete aspects of the project. Therefore, the text uses real examples, real source code excerpts, structural diagrams and descriptions directly associated with the packages, classes and mechanisms actually present in *GenESyS*. Whenever possible, the duplication of long theoretical explanations is avoided, prioritizing operational descriptions, architectural decisions and use and development procedures.

GenESyS is treated here not just as a simulator, but as an infrastructure open to evolution. This characteristic influences the content of the book. Throughout the chapters, the reader will notice that several parts of the system were designed to allow progressive expansion, such as the *plugins* mechanism, the separation between components and *data definitions*, the organization of the kernel, the simulator's internal language and the parser infrastructure. Therefore, the manual is not limited to explaining how to operate a ready-made system, but also how to participate in its technical continuity.

Ultimately, this book is written with an explicitly practical orientation. It is not intended to exhaust all possible project details nor to replace fine reference supplementary documentation such as source code, wiki, repository documentation, or internal system comments. Its role is to offer the reader a short, consistent and sufficiently deep technical path so that they can use *GenESyS* safely and, when necessary, act on its implementation.

It is therefore hoped that this volume will be useful both for those who are taking their first steps with the simulator and for those who want to understand it as a software architecture and development platform. In both cases, the central point remains the same: transforming *GenESyS* into an effective instrument for work, study, experimentation and technological evolution.

To keep the manual's claims bounded, this book uses a controlled evidence vocabulary. *Implementation evidence* means the source or configuration was inspected directly; *startup evidence* means the application or target starts successfully; *functional evidence* means a workflow was exercised and behaved as described; *numerical evidence* means a quantitative result was reproduced or validated; *scientific evidence* means a domain claim was checked against an explicit reference or experiment; and *release readiness* means a human has explicitly approved publication or public exposure. Unless a chapter states otherwise, a claim should be read in the narrowest supported sense available in the repository. In particular, build success and startup success do not by themselves imply functional, numerical or scientific validation, and the *Worker* service remains documented as local or private only; this manual does not

provide public deployment instructions for it unless the project explicitly approves otherwise.

The maturity labels used in the manual follow the same rule: *supported*, *partially validated*, *experimental* and *planned* describe the current evidence level, not an assurance of release quality. The final supported release set is intentionally left open at this stage and remains a human decision.

It should be noted, however, that this work remains under construction and continuous improvement. Chapters, examples, figures, exercises and annexes continue to be revised, condensed and refined editorially. For this reason, this material should not be considered a finalized or stabilized version. So it cannot be published, reproduced, edited, adapted, stored, or distributed, in whole or in part.
Florianópolis, July 2026.

User Manual

1. What is GenESyS

1.1 Introduction

GenESyS is a systems simulator designed to serve, at the same time, as a practical modeling and simulation tool and as a software base for expansion and research. In this book, however, these two dimensions will not be treated simultaneously. The order was deliberately reversed in favor of usage. First, the reader will learn how to work with *GenESyS* as a simulator user. Then, already familiar with the application and models, you will begin to study it as a software system and as an extensible platform.

This choice is important. A common mistake when approaching an academically rich simulation is to start with its most internal aspects: classes, subsystems, parser, *plugins*, execution infrastructure. Although all of this is essential, this is not the best entry point for those who want to really put the system to work. The most productive path begins with use: understanding what the simulator offers, how the user sees it, how models are opened, executed and observed, and how the interface organizes this work.

This chapter, therefore, fulfills a guidance function. It introduces *GenESyS* as a working environment for modeling and simulation, defines the role of the graphical interface as the main point of user interaction, and explains how the rest of the manual is organized. The

objective is not yet to teach how to install the system or use its menus in detail, but to establish a sufficiently clear vision so that the next chapters make sense as a natural continuation.

1.2 Purpose of GenESyS

GenESyS should be understood, firstly, as an environment for working with simulation models. This means that its value to the user is not just in "running" a program, but in enabling a complete cycle of working with models: opening, examining, parameterizing, executing, observing, comparing, recording and improving.

From a practical point of view, this environment needs to support a set of activities that the user performs recurrently:

- load existing models;
- inspect its structure;
- change parameters and properties;
- run the simulation;
- observe the dynamics of the model;
- interpret initial results;
- save modified versions of the work.

GenESyS was designed exactly for this type of use. Instead of being just an isolated program, it presents itself as a workspace where the model is the central object of interaction. The user does not only deal with abstract commands, but with concrete entities from the simulation domain: components, flows, supporting data, simulated time, events, results and complementary representations of the same model.

1.2.1 GenESyS as a modeling and simulation tool

When seen from the user's perspective, *GenESyS* is a modeling and simulation tool. This means that the main question stops being "how was the software implemented?" and becomes "how is the model built, executed and observed within the software?".

This shift in perspective is essential to the first part of this manual. The user wants to know:

- how to get the simulator;
- how to get it up and running;
- how to open example models;
- how to use the graphical interface;
- how to run simulations;
- how to understand what the system is showing.

It is exactly this path that will be followed in the first chapters.

1.2.2 GenESyS as an evolving system

Although this chapter treats *GenESyS* primarily as a user tool, it is important to note at the outset that it is also an evolving software project. This means that, behind the graphical application and the models that the user manipulates, there is an internal infrastructure made up of a kernel, parser, *plugins*, auxiliary applications and test engines.

This dimension, however, should not interfere with the readers initial path. The fact that the system can also be studied and improved as software does not change the learning order proposed in this book. First, the reader will use the simulator. Afterwards, you will study its implementation.

1.3 How the User Works with GenESyS

The user's work with *GenESyS* can be described as a relatively clear flow, although composed of several complementary steps. In a typical situation, the user:

1. obtains the project and prepares the environment;
2. compiles and starts the application;
3. opens the graphical interface;
4. load an already saved model or start a new one;
5. inspects model elements;
6. runs the simulation;
7. observe your behavior and results;
8. saves the work and its variations.

This flow is more important than it might seem at first glance. It defines not only the way the user works with the simulator, but also the organization of the first half of the book. Each of the next chapters will delve deeper into one of these steps.

Table 1.1: User Workflow in GenESyS.

Step	**Practical goal**
Get the system	Access the code, dependencies, and required environment
Compile and start	Run the graphical application
Open a model	Work on a concrete case within the simulator
Explore the GUI	Understand how the interface organizes the model and simulation
Run the model	Observe simulated system dynamics
Inspect results	Read early signals of model behavior and performance
Save and Continue	Preserve work and prepare new iterations

The most important aspect here is that *GenESyS* should not be used as an isolated command system, but as an iterative work environment. The user opens models, explores, mod-

ifies, executes, compares and reexecutes. It is this iterative nature that makes the GUI so important in the initial use of the tool.

1.4 The Graphical Interface as the Main Point of Use

In this first part of the manual, the graphical interface will be treated as the main way of using *GenESyS*. This decision is not just editorial; it arises from the expected usage profile of the simulator user. Those who are working with models, especially in the initial learning phase, need a more direct, more visible and more guided form of interaction than the mere manipulation of internal project artifacts.

The GUI precisely fulfills this role. It organizes the model as a visual and operational object. Through it, the user can:

- open saved models;
- observe its structure;
- browse elements and properties;
- trigger simulation commands;
- monitor system behavior;
- access complementary views of the model, including its textual description.

That's why this book won't start with terminal applications or examples implemented in `C++`. These artifacts are important, but they belong to the developer's field. For the user, the most natural path is the graphical interface and the already saved models.

1.4.1 Why GUI comes before language

Another important decision is the order between GUI and language. The *GenESyS* language, presented to the user through the *Simulang* tab, will only make sense after the reader has already developed a concrete notion of the model in the graphical interface. If the language text is introduced too early, it tends to be perceived as an opaque formalism. On the other hand, after the user has seen the model in the GUI, run its simulation and observed its elements, the textual description starts to function as a second way of reading the same system.

This order is pedagogically superior:

1. first, the user sees and manipulates the model;
2. he then notes how this model can also be described verbatim.

Thus, language enters as a deepening and not as an initial obstacle.

1.5 Example Models as a Gateway

Initial contact with a simulator is rarely more productive when starting from scratch. For most users, the best way to get into the system is to open an existing model, look at how it is

organized, and use that case as a basis for exploration. *GenESyS* follows exactly this logic.

Here, example models play a didactic and operational role at the same time. They are didactic because they help the user to see concrete cases of using the tool. And they are operational because they can already be opened, executed, inspected and modified within the GUI.

By starting with a saved model, the user does not have to face all the initial difficulties simultaneously:

- learn the interface;
- choose components;
- set parameters;
- organize the flow;
- check whether the model is coherent.

Instead, he starts working with a model that already exists and can focus his attention on understanding the simulator. This will be the strategy adopted in the next chapters.

1.6 What the Reader Will Find in This Manual

This book is organized into two clearly distinct parts.

The first part is the *User Manual*. In it, the reader will learn how to:

- get the project;
- prepare the environment;
- compile the application;
- open the GUI;
- work with example models;
- run simulations;
- observe model results and representations.

The second part is the *Developer Manual*. In it, the focus shifts from simulator operation to internal structure. The reader will study:

- the organization of the source code;
- the simulator kernel;
- components and *data definitions*;
- the *plugins* system;
- the parser and internal language;
- auxiliary applications, tools and tests.

This division should not be seen as mere editorial convenience. It corresponds to two real ways of working with the system. First, the reader needs to be able to use the simulator. Then, if you wish, you can understand and modify its implementation.

Table 1.2: Overall structure of the GenESyS manual.

Book part	Main purpose
User Manual	Teach how to use GenESyS as a modeling and simulation application, with an emphasis on the GUI and saved models
Developer Manual	Teach how to understand, extend and improve GenESyS as a software project

1.7 How to Read This Book in the Most Beneficial Way

There are at least two main reader profiles for this manual.

The first is the reader who just wants to use *GenESyS* as a simulator. In this case, the natural reading is linear throughout the first part of the book, with special attention to the chapters on installation, GUI, example models, execution and observation of results.

The second is the reader who, in addition to using the system, intends to study it and improve it as a developer. For this profile, the best strategy is to first also go through the user part. This prevents the internal architecture from being studied too abstractly. A developer who has already seen the GUI in operation and opened real models then better understands the role of the kernel, the parser and the *plugins*.

In other words, the first part of the book is not just preparatory for the user; it also prepares the future developer to read the system from the correct point of view.

1.8 Final Considerations

This chapter introduced *GenESyS* as a modeling and simulation tool, emphasizing the user perspective. The most important point to remember is that the system must initially be understood as a work environment and not as an object of architectural analysis. For the user, the natural path begins with the graphical interface, saved models and the execution of concrete cases. Only then does it make sense to delve deeper into the textual language of the model and, further, into the internal structure of the software.

This order does not reduce *GenESyS*'s wealth; on the contrary, it makes it more accessible. By starting with use, the reader builds a concrete basis for everything that comes later. With this established, the next natural step is to prepare the working environment, that is, obtain the project, install the dependencies, compile the application and achieve the first functional execution of the GUI.

Test your understanding of this content by answering the following questions:

1. Why does this book start by treating *GenESyS* as a tool before treating it as a software project?
2. What is the advantage of adopting the GUI as the main entry point for the user?

3. Why should the *Simulang* tab and template language be presented only after the graphical interface?
4. Why are example models the best gateway for the first practical contact with the simulator?

2. Download, installation and compilation

2.1 Introduction

After understanding the role of *GenESyS* and the way this manual is organized, the reader needs to put it to work. That is the objective of this chapter. Instead of treating compilation as a central theme of the book, it will be treated here as what it really is for the user: a necessary step to achieve graphical application and start working with models.

It is important to note that *GenESyS* is a broader software project than a single application. There are kernel, parser, *plugins*, tests, auxiliary applications and other internal elements. However, this complexity does not need to be absorbed by the user right from the start. The practical goal of this step is simple: obtain the project, prepare the environment, compile the GUI and validate the first execution of the graphical application. If this is achieved safely, the reader will already have what is necessary to advance to the phase of effective use of the simulator.

2.2 Getting the project

GenESyS is available as a source code repository. Therefore, the first step is to obtain a local copy of the project. In an environment with `git` properly installed, this procedure is straightforward.

```
git clone https://github.com/rlcancian/Genesys-Simulator.git
cd Genesys-Simulator
```

Listing 2.1: Initial Clone of the GenESyS Repository

This command produces a local copy of the repository. From then on, the user will have on their machine the set of files necessary to prepare the work environment. It is advisable, at this point, to carry out an initial inspection of the root of the project, just to recognize its presence and structural volume.

```
pwd
ls
```

Listing 2.2: Initial Inspection of the Repository Root

The user does not yet need to interpret in depth everything that appears in this list. Just recognize that there is a source tree, there is a templates folder, and there are configuration files that support the build and the rest of the project.

2.3 What the user really needs at this stage

A frequent difficulty in academic and research projects is assuming that the user needs to understand the entire internal organization before being able to use them. For *GenESyS*, this strategy would be bad. The user of this first part of the book does not need to immediately study the kernel structure, *plugins* or the parser. He only needs enough to:

- prepare a coherent environment;
- compile the graphical application;
- start the GUI;
- confirm that the interface is functional.

Therefore, the organization of the repository should be read, at this point, with only a practical concern. The `source/` folder contains application and system code. The `models/` folder will be important in the next chapters, as it is where the example models used by the user will come from. Other elements of the project may remain in the background for now.

2.4 Environment dependencies

The current version of *GenESyS* uses `CMake` as the main configuration and build system. This means that the user's build environment must offer a minimum set of tools that are compatible with that organization. In addition to `CMake` itself, the project depends on a modern `C++` compiler and a working installation of Qt for the graphical application.

In the documented baseline, the build uses `CMake` 3.24 or newer, the `Ninja` generator, the `C++23` language standard and the Qt6 development packages.

From a practical point of view, it is worth thinking about the environment in three layers:

1. build tools;
2. graphics and support libraries;
3. compiler and general development utilities.

In the first layer, the central element is `CMake`. In the second, the most important point is Qt, especially the modules associated with the graphical interface. The third includes the `C++` compiler and the usual development tools for the chosen platform.

In the current build baseline, the GUI is expected to be compiled against `Qt6`. For the user, this means that the environment needs to make the Qt6 development packages available in a coherent way, together with the build system and compiler required by the project.

2.4.1 Minimum environment checklist

Before starting to configure the build, the user must confirm that they have, at least:

- `git`;
- `cmake`;
- `Ninja`;
- the modern `C++` compiler;
- a working installation of `Qt6`;
- basic terminal and file system tools.

This checklist does not replace specific instructions for each platform, but it prevents the compilation process from starting in a clearly incomplete environment.

2.5 Compiling the graphical application

GUI compilation must be conducted in a build directory separate from the source tree. This practice is recommended for two reasons. The first is to keep the repository clean, without mixing generated files with the original project files. The second is to make it easier to repeat the process in case of adjustments, cleaning or reconfiguration of the environment.

With the build tree kept under the repository root, the user can configure the project. Since the main path of this first part of the book is the GUI, the configuration must explicitly

enable the graphical application. A typical initial configuration is:

```
cmake --preset gui-app
```

Listing 2.3: Initial GUI Build Configuration

This preset configures the `build/gui-app` tree, enables the graphical application and selects the current `CMake/Ninja` baseline expected by the project. Depending on the platform and local environment, the user may want to adjust other options. However, as an initial guideline in the manual, the essential point is to ensure that the GUI is included in the build process. After that, the compilation itself can be carried out.

```
cmake --build --preset gui-app --parallel "$(nproc)"
```

Listing 2.4: GUI Build Compilation

The build preset already drives the aggregate GUI target `genesys_gui`, which depends on the application target `genesys_gui_application`. That application target produces the runtime executable named `genesys-gui`.

2.5.1 Why is the GUI explicitly enabled

It is important to understand the logic of this configuration. The project contains more than one possible application, but the user manual will not be guided by all of them. The target of this first half of the book is the graphical interface. Therefore, the GUI is explicitly enabled in the build configuration. This decision reduces ambiguities and aligns the technical process with the actual way the user will work with the system.

2.6 How to interpret configuration and compilation

At first glance, the textual output of `CMake` and the compilation process may appear excessively detailed. However, the user does not need to interpret everything. There are just a few essential signs to look out for.

In configuration:

- Qt must be localized correctly;
- the process must not end with a fatal error;
- the build directory should be generated normally.

In compilation:

- the process must complete the generation of the GUI executable;
- linking failures or missing libraries should not appear;
- the target of the graphical application must be produced at the end.

This selective reading of build output is important. The user does not yet need to analyze the entire internal chain of project targets. He only needs to confirm that the path to the graphical application was successful.

2.7 What to do in case of an error

Compilation problems are part of working with software, especially when the project is evolving. This does not mean, however, that the user needs to immediately enter into architectural debugging of the system. The most useful approach is to start with the simple causes.

In case of error, first check:

- whether Qt was installed correctly;
- whether `cmake` is available and updated;
- whether the `C++` compiler is correctly configured;
- if the build directory was created clean;
- that there are no residues of previous incompatible configurations.

A particularly useful practice is to remove the `build` directory and repeat the process in a clean environment when inconsistencies that are difficult to interpret appear. In many cases, this eliminates configuration waste or previous poor choices.

2.8 First run of the GUI

Once the compilation is complete, the next objective is to start the graphical application. The generated executable corresponds to the `genesys_gui_application` target, and its runtime name is `genesys-gui`. From the preset build directory, it is available at `build/gui-app/source/appli`

```
./build/gui-app/source/applications/gui/genesys/genesys-gui
```

Listing 2.5: Generic Example of the First GUI Run

The exact path may vary, but the intention is always the same: to reach the main application window and confirm that the GUI is functional.

2.8.1 What to validate on the first run

When starting the application for the first time, the user must check some very specific points:

- the main window opens correctly;
- menus and visible areas of the interface appear coherently;
- there is no immediate structural failure when application begins;
- the GUI appears ready to open or create templates.

There is no need to open models or run simulations yet. The goal of this step is more modest, but fundamental: to confirm that the prepared environment actually leads to a working graphical application.

2.9 First useful observations about the interface

Once the GUI is open, the user can read the application very briefly, without trying to use it in depth. It is worth noting:

- the presence of a menu bar;
- the central work area;
- the existence of panels, trees or side flaps;
- the availability of commands linked to model, editing and simulation.

This observation does not replace the following chapter, which will deal specifically with the graphical interface. However, it helps to transform the first execution into a more concrete experience than simply “the program opened”.

2.10 Preparing the next step

If the reader managed to get this far, then they already have the most important element of this initial phase: a functional installation of *GenESyS* ready for use. This is enough to move forward. It is not necessary, at this point, to delve into the details of the code structure or explore all possible build options. From here, the manual moves from the technical problem of preparing the environment to the practical problem of using the simulator.

In other words, the reader is now in a position to start working with the graphical application itself.

2.11 Final Considerations

The purpose of this chapter was to take the reader from obtaining the project to the first functional execution of the *GenESyS* GUI. Although this step involves downloading, dependencies, configuration, and compiling, its role within the book remains instrumental. The objective was not to make the build the center of the manual, but to remove the technical barrier necessary for the use of the simulator to actually begin.

With the graphical application in operation, the next natural step is to better understand the main window and its organizational logic. Therefore, the following chapter starts to deal with the graphical interface as a user’s working environment, showing how the GUI organizes menus, editing areas, simulation commands, navigation panels and different views of the model.

Test your understanding of this content by answering the following questions:

1. Why is GUI compilation treated in this manual as a means to achieve simulator use, and not as the book's organizing axis?
2. What minimum dependencies does the user environment need to provide before configuring the build?
3. Why is it recommended to build the project in a separate build directory?
4. What signs show that the first run of the graphics application was successful?

3. First steps in the graphical interface

3.1 Introduction

Once the *GenESyS* graphical application is running, the user actually enters the simulator. It is from that moment on that the system stops being just a compiled project and becomes a work environment. This chapter has precisely this function: transforming the first execution of the GUI into a guided first contact with the interface.

The proposal here is not yet to model in detail or run complete examples. Before that, the reader needs to recognize the logic of the main window, understand which areas of the interface are most important, understand where the essential commands are and build a sufficiently clear notion of the workspace that *GenESyS* offers. In other words, this chapter teaches the user how to locate themselves within the application.

This step is more important than it may seem. A rich interface, with multiple menus, tabs, trees, panels and simulation commands, can generate the impression of excessive complexity when the user tries to understand it at once. The best strategy is different: first recognize the global structure of the application, then identify its functional groups and, only then, start working with concrete models. It is exactly this route that will be followed here.

3.2 The Main Window as a Workspace

GenESyS's graphical interface is organized around a main window that centralizes modeling, inspection, simulation, debugging, and reporting operations. This means that the user is not faced with an application with a single panel or a minimal set of isolated commands. Instead, the GUI was designed as a complete work environment in which different tasks can be performed through different views of the same model.

From a usage point of view, this main window should be understood as the space where five fundamental dimensions of working with the simulator are articulated:

1. model management;
2. graphic editing;
3. parameterization and inspection;
4. running the simulation;
5. observation of results and system behavior.

This organization is especially important because the user does not work with the model just before execution or just after it. The model is continually built, adjusted, inspected, and reevaluated. The GUI was structured to support exactly this cycle.

3.3 Opening the Application and First Visual Reading

When starting the graphical application, the user must resist the temptation to immediately click on all available menus and commands. The best way is a guided visual reading of the main window. This reading can be done in very simple steps.

First, look at the menu bar and notice that it organizes work into functional categories. In general, model actions, editing, visualization, simulation, tools and auxiliary information are present. Second, notice the central area of the interface, which is the main graphical representation space of the model. Third, note the existence of tabs, side or bottom panels and navigation trees, which complement the central view. Fourth, note that the application was not designed just to edit a diagram, but to monitor the entire life of the model, including its simulation and inspection.

This first reading is enough to draw an important conclusion: the *GenESyS* GUI is not a simple graphical editor. It is an integrated work interface.

3.4 Interface Functional Groups

The most productive way to understand the GUI is to divide it into functional groups. Instead of memorizing isolated commands, the user should ask: what kind of work does each part of the interface organize?

3.4.1 Template Commands

The first functional group is model commands. They are what allow you to start a new job, open an already saved model, save the current model, close it, check its consistency and obtain information about it. These commands structure the most basic cycle of using the tool.

In practice, this means that the user must recognize at the outset where the actions responsible for:

- create a new model;
- open an existing model;
- save work;
- close the current model;
- check the model before simulating.

These commands will be used repeatedly throughout the first part of the book.

3.4.2 Editing Commands

The second group is editing commands. This includes actions such as undo, redo, copy, paste, delete, group, ungroup and, in some cases, organization and alignment commands. This set reveals that the interface was designed for effective editing of the model, and not just for passive observation.

Even if the user is not yet building new models, it is important to realize right away that the GUI supports modification of graphic and structural elements. This will be useful when the example models start to change, even slightly.

3.4.3 View Commands

Another important group is view commands. They include adjustments such as zoom, grid, guides and other features that facilitate the visual organization of the graphic space. On first contact, the user does not need to explore all these resources in detail, but must recognize them as tools to support reading and editing the model.

In particular, zooming is often useful right from the start, because different models may require different observation scales.

3.4.4 Simulation Commands

A central group of the GUI is made up of commands linked to running the simulation. These typically include starting, pausing, resuming, stopping, step-by-step, and configuring the simulation. These commands show that the application not only organizes the model as a structure, but also monitors its execution.

In this chapter, the reader must only recognize the existence and location of these commands. Its effective use will be deepened when we start working with concrete models. Still,

it's important to stick to the idea that the same GUI used to open and edit the model will also be used to conduct your simulation.

3.4.5 Inspection and Support Commands

In addition to the previous groups, the interface also offers inspection and support mechanisms: property panels, component trees and *data definitions*, debug areas, report tabs and other auxiliary views. The presence of these resources reinforces an essential point: *GenESyS* was not designed just to design models, but to study them in execution.

Table 3.1: Main Functional Groups of the GenESyS GUI.

Functional Group	**Role in the User's Work**
Template	Create, open, save, close, check and organize work with templates
Editing	Changing and rearranging model elements
View	Adjust scale, guides and visual organization of the graphics area
Simulation	Start, pause, resume, stop and track execution
Inspection and support	Examine properties, model elements, results and auxiliary information

3.5 The Graphics Area of the Model

The graphics area is the visual center of working with *GenESyS*. It is where the model appears in a diagrammatic form and it is where most of the user's attention tends to focus when reading and editing the models. At first glance, this area should be understood as the main space where the model's flow becomes visible.

This is important because the user needs to start reading the simulator by the way it shows the model, and not just by the way the program organizes its menus. The graphic area allows you to view:

- the components present in the model;
- the connections between these components;
- the general logic of the modeled flow;
- the visual distribution of elements within the workspace.

Even before opening a concrete model, the user should already perceive this area as the core of the simulator's visual experience.

3.5.1 Graphic area is not just drawing

An important point is that the graphics area should not be confused with a simple drawing editor. What is organized there are not just geometric shapes, but the representation of the model that will later be simulated. In other words, graphic space is also semantic space. Each visible element there tends to correspond to a relevant modeling entity.

This observation helps the user to look at the interface in the correct way. Instead of thinking that he is just manipulating visual objects, he must understand that he is interacting with the operational representation of the model.

3.6 Trees, Panels, and Helper Editors

In addition to the central graphics area, the *GenESyS* GUI offers other elements that are equally important for everyday work. These elements complement the visual reading of the model and make it possible to inspect it in a richer way.

3.6.1 Navigation Trees

Navigation trees help the user locate system and model elements. Depending on the context, they may show:

- *plugins*;
- model components;
- *data definitions*;
- other auxiliary structures associated with the current work.

At first glance, these trees should be seen as guidance mechanisms. They help answer the question: What elements are available or present in the model I'm working with?

3.6.2 Property Editor

The property editor is one of the most important tools in the GUI. It is through this that the user stops just observing the existence of an element and starts dealing with its attributes, parameters and configuration options. In other words, this is where the template starts to become effectively editable.

In practice, this means that the selection of an element in the graphics area or in a navigation tree tends to be associated with the display of corresponding properties. The next chapters, this mechanism will appear continuously, because almost every relevant modification of a model goes through some type of parameterization.

3.6.3 Complementary tabs and views

The presence of central tabs and panels organized by tabs shows that the GUI was built to support multiple perspectives on the same model. Some of these perspectives are visual, others are textual, others are focused on execution, debugging, or presenting results. The user does not need to master all of this now, but must retain the idea that the model is not observed only through a single visual surface.

This multiplicity of views will be especially important when, later on, we introduce the *Simulang* tab as a complementary textual representation to the graphical model.

3.7 Minimal User Guidance Flow in the GUI

In order to familiarize yourself with the GUI to be productive, it is advisable to adopt a short guideline. Instead of randomly exploring the main window, the user can follow this sequence:

1. find template commands;
2. identify the central graphics area;
3. locate auxiliary trees and panels;
4. find the property editor;
5. recognize basic simulation commands;
6. note the existence of additional tabs.

This script has an important advantage: it prevents the richness of the interface from being perceived as clutter. By following a simple order, the user transforms the main window into something intelligible and progressively explorable.

3.8 What Is Not Necessary to Master Now

A successful first contact with the GUI does not require mastering the entire application. It is not necessary, for example:

- use all editing commands;
- understand all debug panels;
- interpret all interface trees;
- know in depth all types of available elements;
- also explore all of the model's languages and representations.

On the contrary, trying to master everything immediately tends to produce noise. The most efficient learning occurs in layers. First, the user understands the general structure of the interface. Then, he starts working with concrete models. Then, it begins to report the GUI with execution, results and textual language. This is the route adopted in this manual.

3.9 Good Practices When First Encountering the Interface

There are some simple practices that make the initial experience with the GUI much more beneficial.

The first is to always work with a concrete objective. Instead of "exploring the interface", try to locate something specific: the open model command, the graphics area, the properties panel or the simulation commands.

The second is not to confuse quantity of resources with difficulty of use. The *GenESyS* interface is broad because it organizes several phases of working with models. This does not mean that all resources need to be used simultaneously.

The third is to look at the interface as an integrated environment. Menus, graphics area, trees, properties, simulation and reports are not independent parts. They form a single workflow.

The fourth is to accept that some parts of the GUI will only make sense when a model is actually loaded. That's why the next chapter will go directly into the example models.

3.10 Final Considerations

This chapter introduced the *GenESyS* graphical interface as the user's main working environment. Instead of treating the GUI as a dispersed set of commands, the text sought to show its organizational logic: the main window brings together resources to manage models, edit them, parameterize them, simulate them and observe them from different perspectives. This understanding is essential because it prevents the user from reducing the application to a simple graphical editor or, at the opposite extreme, perceiving it as an excessively complex and cluttered interface.

With this guidance in place, the next natural step is to start working on concrete models. This is when the graphical interface really takes on operational meaning. Therefore, the following chapter will deal with the example models already available in the project and their initial use in the GUI, allowing the reader to open, inspect, execute and modify real simulation cases.

Test your understanding of this content by answering the following questions:

1. Why should the *GenESyS* GUI be understood as an integrated work environment, and not just as a graphical editor?
2. What are the most important functional groups of the interface in a first contact?
3. What is the function of the model's graphical area within the simulator usage experience?
4. Why isn't it necessary to immediately master all the application's menus, trees and panels?

4. Example models and initial use of the simulator

4.1 Introduction

Once the *GenESyS* graphical interface is up and running and the reader has an initial view of your organization, the next natural step is to start working with concrete models. This is the point at which the simulator stops being just an application open on the screen and starts to become an effective modeling and experimentation environment. To achieve this, the most appropriate path is not to start immediately by creating a complete model from scratch, but to use the example models already available in the project.

This decision is important from both a didactic and practical point of view. From a didactic point of view, an example model already offers a coherent structure, with interconnected elements, completed properties and expected behavior. This reduces the initial cognitive load on the user, who does not need to simultaneously learn the interface, the semantics of the components, the flow construction logic and the simulation execution mechanism. From a practical point of view, example models allow you to check early on whether the system installation is functional and whether the GUI can open, display, interpret and execute real models.

In this chapter, the objective is to teach the reader how to use the models saved in the `models/` folder as a gateway to the simulator. At the end of reading, the user should know

how to locate an example model, load it into the graphical interface, read its visual structure, identify the main flow of the modeled system, run the simulation and make small controlled modifications to observe the effect of these changes.

4.2 Why start with example templates

Rich simulations tend to offer many job possibilities from the first contact. This wealth, although valuable, can become an obstacle when a good entry strategy is not chosen. In the case of *GenESyS*, using example templates solves exactly this problem.

When opening an already saved model, the reader has immediate access to an artifact that:

- is already organized coherently;
- already contains a flow logic or modeling structure;
- can now be inspected by the GUI;
- can now be executed;
- can now be used as a basis for tracked changes.

This allows learning to occur in layers. First, the user learns to recognize the model as a visual object within the interface. Then learn how to execute it. Then, begin relating the graphic elements to their effects in the simulation. Only later did it begin to produce its own models with greater autonomy.

In pedagogical terms, the example model fulfills a role similar to that of an experiment already set up in a laboratory: it does not replace active learning, but creates a stable initial context in which the user can begin to observe, formulate hypotheses, modify parameters and interpret results.

4.2.1 The mistake of starting from scratch too soon

There is a natural temptation, especially in technically curious readers, to try to build their own model during their first interactions with the system. Although this initiative may seem productive, it often introduces unnecessary difficulties.

When the user starts too early from an empty space, he needs to decide at the same time:

- which elements to insert;
- in what order to connect them;
- which properties to adjust;
- what supporting data is needed;
- how to check if the model is coherent;
- what to expect from execution.

This accumulation of decisions makes it more difficult to distinguish what is a difficulty inherent to the modeling and what is simply an initial lack of knowledge of the tool. The

example model avoids this noise. It separates the problem of using the interface from the problem of full model authoring.

4.2.2 Example templates as reading material

In addition to serving as executable cases, the example templates should also be read. This reading, however, is not the same as reading a text or a source code. It involves:

- observe the visual arrangement of the model;
- perceive the main flow sequence;
- identify entry and exit points;
- recognize auxiliary elements that support execution;
- report visual structure to observed behavior.

With this, the model stops being a file and becomes an object of study within the GUI.

4.3 The `models/` folder as a reference for use

For the *GenESyS* user, the `models/` folder should be understood as one of the most important areas of the project. This is where the saved models that will be used as an entry point for exploring the simulator are located. In this first part of the book, this folder has much higher priority than any source code directory.

The role of this folder is twofold. First, it concentrates concrete material for using the simulator. Second, it provides the user with a repertoire of examples from which to learn how to open, read, run, and change models. This means that for initial learning purposes, the user does not need to worry about software implementation details. Your main universe of work is precisely in these model files.

4.3.1 What to look for in the `models/` folder

When browsing the `models/` folder, the user must look for models that preferably satisfy three criteria:

1. are visually simple or moderately simple;
2. have an easily identifiable main flow;
3. allow an execution whose behavior can be observed without great interpretative effort.

This doesn't mean that larger or fancier models aren't valuable. It just means that, when entering the simulator, it is advisable to prioritize cases that favor clarity and readability.

4.3.2 How to select good first examples

As a first approximation, the best examples tend to be those where the user can answer the following questions relatively quickly:

- where does the flow begin?
- what main elements does it go through?
- where does it end?
- Are there queues, resources, attributes, or auxiliary data that support this behavior?
- what do I expect to see when this model is run?

If these questions can be answered without great difficulty, the model is probably a good candidate for initial exploration.

4.4 Opening a sample model in the GUI

With the application already started, the next step is to open a real model. The operation of opening a model file should not be treated as a simple technical detail, because it inaugurates the phase in which the GUI starts to operate effectively on a concrete work object.

The general procedure is as follows:

1. start the *GenESyS* GUI;
2. find the menu or command corresponding to opening the model;
3. navigate to the `models/` folder;
4. select an appropriate template file;
5. confirm the opening and observe the interface update.

After that, the central graphics area should start displaying the model, and the GUI panels will tend to reflect the loaded elements, allowing for more detailed inspection.

4.4.1 What to confirm immediately after opening

Once the template is loaded, the user must confirm a few simple but important points:

- the graphics area has been filled with the model structure;
- there are visible elements and connections;
- the interface seems to recognize the model as open;
- inspection and simulation commands became usable;
- the user can visually navigate the loaded content.

This confirmation is useful because it distinguishes two very different states of the application:

- the GUI is empty or without an active model;
- the GUI effectively working on a model.

Only in the second case does it make sense to proceed to reading and execution.

4.4.2 When the model appears “too big”

In some cases, the first model opened may appear visually too dense. If this occurs, the user should not immediately interpret this sensation as a problem with the simulator or model. In general, this just indicates that:

- zoom needs to be adjusted;
- the chosen model was not the most suitable for the first exploration;
- reading must start with a main region, and not with the entire scene.

In this situation, the best practice is to return to a simpler example or restrict the analysis to the most easily identifiable main flow.

4.5 How to read a model in the graphics area

Once the model is loaded, the user’s next task is to read it. This reading is neither purely visual nor purely technical. It is a reading guided by questions about behavior and structure.

The best initial reading order is:

1. find the beginning of the flow;
2. follow the main path of the model;
3. identify the end of the flow;
4. observe support elements;
5. only then examine local properties and details.

This order is important because it prevents the user from getting lost in details before understanding the central logic of the model.

4.5.1 Identifying the main flow

The main flow is the most important structure of the model for first contact. It corresponds to the essential path taken by the main entities, events or transformations represented in the system. In many cases, this flow can be recognized by a sequence of components connected in a relatively linear way or by a visual arrangement that clearly suggests the progression of the modeled process.

The question that should guide the user is:

What is the minimal story this model is telling?

When this question begins to be answered, the model stops being a set of elements on the screen and starts to be perceived as an organized system.

4.5.2 Distinguishing main flow from auxiliary elements

A careful reading of the model depends on distinguishing what constitutes its central path from what supports it. Often, in addition to the main flow, the model also involves auxiliary data and structures: queues, resources, statistics, attributes, variables, counters or other definitions.

The user should not ignore these elements, but he should not start with them either. The correct procedure is:

1. understand the flow first;
2. then ask what support elements allow this flow to work;
3. only then observe specific properties.

This order greatly improves the readability of the model.

4.6 Running the model for the first time

After opening and reading the model, the decisive moment comes: running the simulation. It is at this stage that the user checks whether the observed visual structure actually corresponds to understandable dynamic behavior.

On first use, execution should be conducted with exploratory intent and not in a rush. The objective is not just to "run" the model, but to understand:

- if the simulation starts correctly;
- whether the model's behavior makes sense in relation to its structure;
- whether the GUI provides sufficient signals to track progress;
- whether the model reaches a final state or evolves coherently.

4.6.1 What to watch out for while running

When running a model for the first time, the user should avoid trying to read all output and all panels at the same time. It's best to focus on a few core aspects:

- there is a clear indication that the simulation has started;
- the model appears to visually evolve or produce coherent signs of progress;
- the observed behavior is compatible with the previously identified main flow;
- there is no obvious structural flaw in the execution process.

In other words, the first run serves to connect the static model to its dynamics.

4.6.2 First validation of behavior

A well-done initial validation is modest but sufficient. The user must be able to respond:

- did the model open correctly?
- Could the simulation be started?

- does the observed behavior seem plausible?
- Did the execution finish or progress without obvious inconsistencies?

If these answers are positive, then the first practical contact with the simulator has already been successful.

4.7 Modifying an example model in a controlled way

After opening and running a model, the next most productive step is to make small, controlled modifications. This step is pedagogically very important, because it transforms the example model into an active learning instrument.

The best initial modifications are those that:

- do not completely change the structure of the model;
- can be easily reversed;
- produce an observable effect;
- help the user understand the function of properties or parameters.

Examples of appropriate changes at this stage include:

- adjust a simple property;
- change a name or label;
- change a numerical value associated with a component;
- save a modified copy of the template.

4.7.1 Why small changes are better

If the first modification is too big, the user loses the ability to report cause and effect. It is difficult to know whether the observed change in behavior is due to a specific parameter, an extensive structural change or an inconsistency introduced accidentally.

Small changes, on the contrary, allow the user to experience the simulator logic almost like in a laboratory:

1. observe the original state;
2. change a specific point;
3. rerun;
4. compare the effect.

This methodology is much more useful than making extensive modifications at the beginning.

4.8 Best practices for exploring example models

A few simple practices make initial use of the example templates much more beneficial.

4.8.1 Start with smaller cases

Smaller models generally allow for clearer reading of the interface, flow, and behavior. They also facilitate the association between visual structure and observed execution.

4.8.2 Save separate copies

When modifying a sample template, it is recommended that you save a new copy rather than immediately overwriting the original file. This preserves the reference example and makes later comparisons easier.

4.8.3 Use execution as a reading tool

Often, the model only becomes fully intelligible when it is executed. Therefore, execution should not be seen just as a final step, but as part of the process of understanding the example itself.

4.8.4 Return to structure after running

A very useful practice is to re-read the model visually after the first run. Observed behavior often sheds new light on the function of its elements, connections, and supporting data.

Table 4.1: Recommended workflow for exploring example models in GenESyS.

Step	**Goal**
Open the model	Load a working case in the GUI
Read the main flow	Understand the central logic of the modeled system
Recognize supporting elements	Understand what data and structures support the flow
Run the simulation	Report visual structure to observed behavior
Change something simple	Turn the example into an active learning object
Rerun and compare	Observe the effect of the modification made

4.9 Final Considerations

In this chapter, example models have been presented as the user's main entry point into practical work with *GenESyS*. Instead of starting with the creation complete of a new model, the reader was instructed to open already saved cases, read them visually, identify their main flows, execute them and make small controlled changes. This strategy allows learning to happen in a progressive and concrete way, without requiring the user, right from the beginning, to fully master the entire tool.

The essential point to remember is that an example model should not be treated as a simple passive demonstration. It is, at the same time, an object of reading, an object of execution

and an object of experimentation. From here, the user is not only familiar with the GUI as a structure, but begins to use it effectively on real models. The next natural step is to deepen the relationship between graphical interface, execution, behavior observation and textual representation of the model, which will be done in the next chapter by introducing the *Simulang* tab, the execution observation mechanisms and the first procedures for analyzing the system's behavior.

Test your understanding of this content by answering the following questions:

1. Why are example templates a better entry point than immediately creating a complete template from scratch?
2. What is the most productive order to read a model loaded in the GUI?
3. What should the user validate when first running a sample model?
4. Why is it important to initially modify only small aspects of the model?

5. GUI, Simulang, model execution and observation

5.1 Introduction

In the previous chapters, the reader got *GenESyS* up and running, got to know the general logic of the graphical interface and started working with example models already saved in the project. From this point forward, it becomes possible to take an important step: move away from just a structural reading of the models and start observing them in execution, following their dynamics and relating the graphical representation to the textual representation of the system.

This chapter has exactly that purpose. It delves deeper into GUI usage in three complementary directions. First, it shows how the execution of the model should be followed by the user, not as an opaque operation, but as an observable process. Second, it introduces the *Simulang* tab as a textual form of representation of the same model that has already been seen in the graphical interface. Third, it guides the observation of the first results, traces and signs of the system's behavior, so that the user begins to build a more informed reading of the simulation.

The importance of this stage is great. A really useful simulator is not only able to open models and run them, but to allow the user to understand what is happening. The *GenESyS* GUI was designed to offer different perspectives of the same work: a graphical perspective, a

textual perspective and a dynamic perspective, linked to execution. From here, the reader will begin to articulate these three layers.

5.2 From static structure to model dynamics

When the user opens a model and looks at its graphical structure, he sees a static description of the system. The components, connections, supporting data and graphic elements indicate how the simulation was organized, but do not yet show, by themselves, the effective temporal behavior of the model. Running the simulation transforms this static structure into a dynamic process.

This point is central to understanding the simulator. A model is not just a well-organized diagram; he is a behavior machine. The function of the simulator is precisely to activate this machine, driving the advancement of simulated time, the processing of events, the circulation of entities and the updating of the system's relevant information.

For the user, the main consequence is this: execution should not be seen as just pressing the start button. It should be understood as the passage from the model description to its operational dynamics.

5.2.1 What changes when the model starts running

When the simulation starts, the user starts working with a new type of reading. Before, the main question was:

> How is this model organized?

During execution, this question changes to:

> How does this model behave over time?

This means observing:

- whether entities are being created, transformed or removed;
- if the flow actually goes through the elements that the model suggests;
- whether resources, queues or supporting data appear to be used coherently;
- if the global behavior of the simulation makes sense in relation to the previously read structure.

5.3 Execution commands and their usage logic

The *GenESyS* GUI provides a set of commands directly linked to the simulation. Among them, the commands to start, pause, resume, step by step, stop and configure stand out. For the user, these commands should not be perceived just as operational buttons, but as different ways of relating to the dynamics of the model.

5.3.1 Start the simulation

The simulation start command is the transition point between the static model and observed behavior. Before activating it, the user must ensure that:

- the correct model is loaded;
- the visual structure makes sense;
- the system appears to be ready to run;
- the objective of the observation is minimally clear.

This last observation is important. It is better to run the simulation with some reading hypothesis for example, observing a specific flow or the effect of a certain property than simply starting the execution without knowing what you want to track.

5.3.2 Pause, resume and advance step by step

One of the great advantages of a rich GUI is that it allows different rates of observation of the simulation. In some situations, the user just wants to check the overall behavior. Other times, you want to look at specific parts of the stream more closely. It is in this context that commands such as pause, resume and step by step become particularly valuable.

These commands transform the simulation into an inspection object. Instead of just letting the system run to completion, the user can:

- temporarily stop execution;
- analyze the current state;
- resume simulation;
- or advance in a controlled manner.

This mode of use is especially useful in learning models, in initial debugging and when reading the behavior of elements whose function is not yet completely clear to the user.

5.3.3 Stop and configure

The stop and configuration commands also deserve attention. Stopping the simulation is not just ending a process, but stopping the evolution of the model in a controlled way. Configuring the simulation means adjusting execution conditions before starting the experiment.

Although the depth of these parameters depends on the model and the context, the user must remember that the execution is not completely rigid: it can be prepared, started, observed and stopped according to different needs.

5.4 Observing model behavior during execution

Observation of execution must be conducted in a guided manner. If the user tries to simultaneously follow all the GUI signals, the entire reading will become fuzzy. The ideal is to observe

the model in levels.

5.4.1 First level: the global flow

At the first level, the user must observe the behavior of the model globally. The most important questions are:

- did the simulation start correctly?
- does the main flow seem coherent with the model structure?
- does the system evolve plausibly?
- does termination of execution make sense?

This is a macroscopic reading. She doesn't look for fine details, but overall consistency.

5.4.2 Second level: specific elements

After observing global behavior, the user can focus on specific elements of the model. This may include:

- a component whose function is not yet clear;
- a point in the flow where the behavior seems particularly important;
- a relevant queue, resource or supporting data;
- a property whose effect has recently been modified.

This localized reading is more productive after the general dynamics have already been minimally understood.

5.4.3 Third level: comparison between runs

An especially useful practice is to compare the model run before and after small changes. This procedure helps the user understand not only how the system works, but also how it will react to changes. The GUI, in this sense, is not just an observation interface, but also a comparison laboratory.

5.5 The *Simulang* Tab as a Textual Representation of the Model

After the reader has already seen the model in the GUI and observed it in execution, the textual representation starts to make much more sense. This is exactly why the *Simulang* tab appears now, and not before.

The function of the *Simulang* tab is not to replace the GUI, but to offer a second way of reading the same model. In this way of reading, the user stops seeing only the graphical organization of the system and starts seeing a corresponding textual description. This is valuable for several reasons:

- makes the model more formally explicit;

- allows you to compare the visual representation with the textual representation;
- helps consolidate understanding of the model;
- paves the way for later chapters on language and development.

5.5.1 How to Read the *Simulang* Tab

Reading the *Simulang* tab must be done with the same didactic caution used in the rest of the first part of the book. The user does not need, at this point, to master the entire syntax or semantics of the language. What he needs is to accomplish that:

- the textual description corresponds to the model you have already seen in the GUI;
- visual elements of the model have a textual expression;
- the language offers a more formal view of the modeled system.

The most productive way to start this reading is to compare excerpts from the textual representation with already known elements of the graphical interface. Instead of trying to decipher the entire text at once, the user should ask:

What part of the graphical model does this region of the text seem to describe?

5.5.2 Why textual representation is useful to the user

Even if the user continues to work primarily through the GUI, the textual representation offers concrete benefits. It helps to:

- consolidate understanding of the model;
- make the system structure more explicit;
- understand the model beyond its visual layout;
- start thinking about the relationship between interface and language.

This experience is particularly valuable because it shows, from an early stage, that *GenESyS* does not treat the model just as a drawing, but also as a formal description.

5.6 Tracing, breakpoints and initial observability

An important feature of a well-structured simulator is to offer mechanisms for observing the behavior of the running system. In *GenESyS*, this is expressed through different means available in the GUI, including traces, events, reports, and controlled stopping or tracking mechanisms.

At this stage of the manual, the reader does not yet need to deeply master these mechanisms. But you need to understand their general function: they make the simulation more observable.

5.6.1 Tracing as reading behavior

Tracing should be seen as a way of recording what happens in the simulation. It helps the user follow the model not only through the visual perception of the interface, but also through an operational narrative of what is happening.

This is especially useful when the model's behavior is not immediately evident from the graphics area alone. In such cases, tracing answer helps:

- what events are occurring;
- in what order;
- with what apparent effects on the state of the system.

5.6.2 Breakpoints as a controlled observation tool

Breakpoints, when used, allow you to interrupt the simulation at points of interest. For the user, this turns the execution into an examineable object. Instead of letting the model evolve completely without intervention, you can stop at relevant states and take a closer look at what is happening.

This tool is particularly useful when:

- if you want to study the behavior of a specific part of the model;
- if you want to validate the effect of a modification;
- if you want to better understand the dynamics of a specific element.

5.7 First reading of the results

In addition to observing the execution in progress, the user needs to start developing the habit of interpreting the results produced by the simulator. Here again, the guidance must be progressive.

The first reading of results should not be statistically sophisticated or overly ambitious. She must answer simple questions:

- did the simulation behave as expected?
- were there coherent signs of progression and termination?
- Did simple changes to the model produce observable effects?
- Do the reports or signals presented by the GUI seem compatible with the model structure?

Only after this initial stage does it become useful to move on to more technical readings.

5.7.1 Results as continuation of model reading

An important point is that the results should not be treated as something separate from the model. They are a continuation of reading the model by other means. The graphical model

shows structure, the execution shows dynamics, and the results show observable consequences of that dynamic. These three dimensions must always be articulated.

5.8 Best practices for observing running models

Some practices make observing the much simulation more beneficial.

5.8.1 Set an observation focus

Before starting execution, try to decide what to observe first: the global flow, a specific component, an effect of a certain change, or the relationship between GUI and Simulang. This prevents dispersion.

5.8.2 Switch between views

The strength of the *GenESyS* GUI is in offering multiple perspectives on the same model. Switch between:

- graphic structure;
- observed execution;
- traces or events;
- textual representation in *Simulang*.

This alternation often produces deeper understanding than any single view.

5.8.3 Make small changes and compare

Whenever possible, make small, controlled changes to the model and compare the behavior before and after. This strategy turns observation into a guided experiment.

5.8.4 Don't try to exhaust everything in a single run

A simulation rarely reveals everything the model can teach in a single run. It is better to read successively, each with a clearer focus, than to try to absorb everything at once.

5.9 Final Considerations

This chapter deepened the use of the *GenESyS* GUI in an essential direction: the articulation between the model's visual structure, simulation execution, behavioral observability and textual representation in *Simulang*. The main idea that the reader should remember is that these dimensions do not compete with each other. They complement each other. The graphical model allows you to see the organization of the system, the simulation allows you to observe

Table 5.1: Recommended Strategy for Observing Running Models in GenESyS.

Step	**Goal**
Observe the graphical structure	Understand the static organization of the model
Start simulation	Transform structure into dynamic behavior
Monitor global flow	Check overall consistency of execution
Explore tracing and events	Make behavior more observable
Read the tab *Simulang*	Relate the graphic model to its textual description
Compare runs	Understand the effect of small model changes

its dynamics, the tracing and pausing mechanisms make this behavior more readable, and the *Simulang* tab offers a textual way to consolidate this understanding.

With this, the first part of the manual begins to gain real density of use. The reader no longer just opens the GUI and loads a model; he began to observe the simulator in operation and report its various representations. The natural next step from here is to consolidate the use of the system with more guided examples and prepare the transition to a more complete understanding of the relationship between modeling, simulation and the internal structure of GenESyS.

Test your understanding of this content by answering the following questions:

1. Why should the execution of a model be understood as a transition from the static structure to the system dynamics?
2. What is the use of commands such as pause, resume and step-by-step during the simulation?
3. Why should the *Simulang* tab only be introduced after the user has already seen and executed the model in the GUI?
4. How do tracing, breakpoints and results complement visual observation of model execution?

II Developer Manual

6. Source Code Organization and Overall Architecture

6.1 Introduction

From this point on, the book explicitly enters its second half: the developer's manual. Changing perspective is important. So far, the focus has been on using *GenESyS* as a modeling and simulation tool, with an emphasis on the GUI, example models and observation of system behavior. Now, the focus becomes understanding the software itself: its internal organization, its central subsystems, the way the code is distributed and the points from which a developer can begin to study, modify and expand it.

Before studying the core of the simulator or discussing *plugins*, parser and applications, it is necessary to build an overview of the repository organization. Without this vision, the developer tends to get lost in files and classes without realizing the architectural function of each region of the code. The goal of this chapter is precisely to avoid this problem. Instead of immediately presenting local details, it organizes the overall project map.

The central idea is simple. *GenESyS* should not be read as an amorphous set of `C++` files, but as a system divided into areas with distinct responsibilities. Some of these areas underpin the core of the simulation. Others organize the extensibility mechanism. Still others implement applications, tools and tests. Understanding this division is the first step to a careful

technical reading of the project.

6.2 From Simulator Application to Software System

In the first half of the book, *GenESyS* appeared to the reader mainly as an application. The user opened the GUI, loaded models, ran simulations, and observed results. For the developer, however, this vision is just the surface of a broader system. The graphic application does not exhaust the project; it is one of its forms of presentation.

This observation is important because it helps to correctly reposition the object of study. The developer does not work just on a graphical interface, nor just on a set of models. He works on software that:

- maintains a simulation kernel;
- represents models internally;
- manages events, entities and simulated time;
- interprets expressions and simulation language;
- loads and integrates *plugins*;
- supports different applications on the same basis;
- provides infrastructure for testing and evolution.

Therefore, the starting point of the second half of the book is not the GUI itself, but the system of which the GUI is a part.

6.3 The Overall Logic of the Directory Tree

The *GenESyS* repository should be read as a tree organized by technical responsibility. Not all directories have the same importance for the developer, and a good initial reading consists precisely in distinguishing what is structural from what is just contextual.

Broadly speaking, there are three main areas of interest:

1. the main source code zone;
2. the zone of usage models and artifacts;
3. the development, build and testing support area.

Within the first of these zones, the `source/` folder contains the effective core of the software. It is the one that starts to dominate the reading from this chapter forward. The `models/` folder, which in the first half of the book served as the main entry point for the user, remains important, but now changes its role: it is no longer just material for use but also becomes material for architectural observation, as it helps the developer to relate the representation of the model to the behavior of the code.

6.3.1 Why the `source/` folder is the center of reading

For the developer, the `source/` folder is the heart of the project. This is where the fundamental subsystems of *GenESyS* are located:

- kernel;
- parser;
- *plugins*;
- applications;
- tools;
- tests.

This division already suggests a first architectural reading of the system. The kernel represents the central infrastructure of the simulation. The parser takes care of the language and expressions. The *plugins* mechanism materializes the extensibility mechanism. The applications expose concrete ways of using the infrastructure. The tools complement the ecosystem. Tests organize the validation of software behavior.

6.3.2 Why `models/` remains relevant to developers

In the first part of the book, the models were treated as material for using the simulator. Now, they also start to serve as software reading instruments. This is because the developer constantly needs to report two things:

- the way the system presents itself to the user;
- the way the system is implemented internally.

The models in `models/` help exactly with this bridge. They show the type of artifacts that the application manipulates and help to report the structure of the models to the set of kernel classes and mechanisms.

6.4 The large areas of `source/`

Reading `source/` must start with its large areas, and not with isolated files. This approach reduces noise and helps build an architectural view before reading detailed classes.

6.4.1 `source/kernel`

The `source/kernel` folder contains the simulator's central infrastructure. It is there that the developer finds the most fundamental elements of the system: model representation, entities, events, simulation mechanisms, utilities and statistics. This is, without a doubt, the most important region of the project for anyone who wants to understand GenESyS in depth.

From an architectural point of view, the `kernel` folder organizes what the simulator core does independently of any specific interface. Graphical applications, terminal applications, tools and other modules can only exist because this core provides them with a stable base.

6.4.2 `source/parser`

The `source/parser` folder brings together the infrastructure associated with interpreting the simulator's language and expressions. It is especially important because many model parameters are not just literal values, but textual expressions that need to be interpreted within the context of the model and simulation.

In the first part of the book, this dimension appeared indirectly in the *Simulang* tab. Here, it becomes clear that the `source/parser` folder should be understood as a software subsystem responsible for connecting text, model semantics, and execution.

6.4.3 `source/plugins`

The `source/plugins` folder organizes one of the most characteristic aspects of GenESyS: its extensibility. Instead of concentrating all functionality in a closed core, the project foresees the incorporation of specialized modules that expand the components, data and other capabilities of the system.

For the developer, this means that an important part of the simulator's evolution happens on this border between kernel and *plugins*. Many of the modeling elements most visible to the user are provided precisely by this mechanism.

6.4.4 `source/applications`

The `source/applications` folder contains applications built on top of the system core. This is where GenESyS stops appearing just as infrastructure and starts to take on concrete forms of interaction, such as the GUI and other auxiliary applications. This layer is important because it shows a valuable architectural separation: the simulation core is not confused with a single application.

This means that the software can be read at two levels:

- as infrastructure;
- as a set of applications on this infrastructure.

6.4.5 `source/tools`

The `source/tools` folder brings together tools that support the use, development or evolution of the system. Although they are not always the first reading priority, these tools can become important in debugging, support, integration, or maintenance contexts.

6.4.6 `source/tests`

The `source/tests` folder represents the project validation dimension. It is especially important for the developer because it offers a concrete path to verify expected behavior, prevent regressions and provide security for changes to the system.

In evolving projects, the existence of an organized test tree is one indication of disciplined development. In GenESyS, this is particularly relevant because the software has multiple layers and extensibility mechanisms.

6.5 A Layered Reading of the System

A very useful way to understand the general architecture of *GenESyS* is to imagine the system in layers.

At the deepest layer is the kernel. It is what defines the fundamental structures and services of the simulation. On top of this layer, language and extensibility mechanisms appear, such as parser and *plugins*. Further up, concrete applications emerge, such as the GUI, which present the user with a way of interacting with this infrastructure. In parallel, tests and tools help support the maintenance and evolution of the project.

This layered reading helps the developer avoid a common mistake: confusing what is structural with what is just surface use. The GUI is very important, but it is not the heart of the system. Likewise, a plugin may be central to a specific modeling domain, but still depends on the kernel to exist.

Table 6.1: Layered Reading of the Overall GenESyS Architecture.

Layer	**Main Function**
Kernel	Central simulation infrastructure, model representation, events, entities, time, statistics and fundamental services
Parser and extensibility	Expression interpretation, simulator language and functional extension by *plugins*
Applications	Concrete forms of interaction with the system, such as the graphical interface
Tools and Tests	Support for development, validation, maintenance and project evolution

6.6 The Role of the `Simulator` Class in the System View

The `Simulator` class provides a particularly useful synthesis of this overview. It presents itself as the highest level class of the simulation kernel and exposes, through direct access, several central system managers. This includes, but is not limited to, `PluginManager`, `ModelManager`, `TraceManager`, `ParserManager`, and `ExperimentManager`.

This observation is valuable because it shows, in a very concrete way, that the GenESyS architecture is not organized around a single file or a single operational class, but around a core that coordinates multiple specialized subsystems. In other words, the developer must un-

derstand `Simulator` as an access point to the simulator ecosystem, and not just as an isolated technical object.

6.6.1 Why This Matters for Reading the Code

If the developer enters the project only through the GUI or a specific modeling component, they run the risk of reading the system in a fragmented way. The presence of the `Simulator` class as an aggregator of managers helps to refocus the reading: software is better understood when seen as a set of coordinated subsystems, and not as a collection of independent parts.

This insight will be particularly important in the next chapter, when the focus shifts to the kernel and classes that effectively control the life of the model in simulation.

6.7 Misreadings This Chapter Avoids

This chapter aims to avoid some common misreadings in projects like GenESyS.

The first is to imagine that the GUI is the entire system. It is fundamental for the user, but it is just an application on a broader infrastructure.

The second is to imagine that the kernel can be read without relation to the parser, *plugins* and applications. Although the kernel is the foundation, it does not live in isolation.

The third is to start studying the code with random files. This practice often produces local familiarity but not architectural understanding.

The fourth is to treat the `models/` folder as something irrelevant to the developer. In fact, it is important because it helps to relate the implementation to the simulator's real work object.

6.8 Recommended Reading Strategy for the Code

The most productive way to study the *GenESyS* source code is progressive.

First, build an overview of the project tree and the responsibilities of its major areas. Then, enter the kernel and study the central classes that articulate the model, simulation, events and observability. Then move on to the component base classes, *data definitions*, parser, and *plugins*. Only then does it make sense to delve deeper into specific applications, tools and tests.

This order is not arbitrary. It follows the softwares own logic:

1. first, the infrastructure;
2. then, the expansion mechanisms;
3. then, the concrete applications;
4. finally, the maintenance and validation instruments.

6.9 Final Considerations

This chapter reorganized the reader's perspective on *GenESyS*. From this point onward, the project is no longer seen just as a graphical application and instead becomes a software system distributed across areas with different responsibilities. The most important point to remember is that the GenESyS architecture must be read in layers: kernel, parser, *plugins*, applications, tools, and tests. Without this overall view, the study of individual classes tends to become fragmented and unproductive.

With this base established, the next step is to enter the most important region of the project: the core of the simulator. It is here that the model, the simulation, the events, the simulated time, the list of future events and the central mechanisms for observing the system's behavior are defined. Therefore, the following chapter will focus on the kernel itself, with an emphasis on the classes that articulate the dynamics of the simulation.

Test your understanding of this content by answering the following questions:

1. Why should the `source/` folder be treated as the center of the project's technical reading?
2. What is the advantage of interpreting GenESyS in layers, rather than reading it as a scattered set of files?
3. Why shouldn't the GUI be confused with the entire system?
4. What role does the `models/` folder still play for the developer, even in the second half of the book?

7. Simulator Kernel

7.1 Introduction

If the previous chapter organized the general map of the project, this chapter goes into the most important region of that map: the core of the simulator. This is where *GenESyS* stops being seen just as an application and starts to be understood as a system that represents models, schedules events, controls repetitions, maintains simulated time and coordinates the observability of the experiment.

The study of the kernel is decisive for a simple reason: everything the user does in the GUI and everything the developer adds via parser, *plugins* or applications ultimately depends on the infrastructure defined here. It doesn't matter whether the model was opened visually, described textually or constructed by code. In all cases it will need to be represented, verified, executed and observed by core kernel classes.

In this chapter, the focus will be on a set of particularly important classes and structures:

- `Simulator`;
- `Model`;
- `ModelSimulation`;

- `OnEventManager`;
- list of upcoming events;
- current entities and events;
- simulation cycle control mechanisms.

The goal is not yet to exhaust all kernel classes, but to build a firm understanding of the core dynamics of GenESyS.

7.2 The Kernel as Simulation Infrastructure

The *GenESyS* kernel should be understood as the layer that makes it possible for the simulator to exist regardless of the interface used. The GUI, terminal application, textual templates, *plugins* and tools may vary; the kernel, however, is the basis that supports the representation of the model and the execution of the simulation.

In architectural terms, this means that the kernel needs to solve at least four fundamental problems:

1. how to represent a simulation model;
2. how to represent the execution state of this model;
3. how to advance in simulated time through events;
4. how to make the simulation observable and controllable.

These four problems appear clearly in the project's central classes. That's why studying the kernel is not studying "another package" of the system, but the part that defines the essential behavior of the simulator.

7.3 The `Simulator` class as kernel entry point

The `Simulator` class occupies a special position in the kernel view. It presents itself as the main class of the simulation system and provides access to important managers such as `PluginManager`, `ModelManager`, `TraceManager`, `ParserManager` and `ExperimentManager`. This organization already reveals something important: GenESyS was not structured around a single monolithic class, but around a nucleus that coordinates specialized subsystems.

From a developer's perspective, the `Simulator` class is useful for two reasons. First, because it offers a synthetic view of the project's architecture. Secondly, because it works as a high-level access point to the other mechanisms of the system. Instead of entering the software in isolated fragments, the reader can start by realizing that there is a central class that articulates the simulator's ecosystem.

7.3.1 What This Class Suggests About the Architecture

The mere presence of managers accessible from `Simulator` already indicates five architectural pillars:

- the system manages models;
- the system manages *plugins*;
- the system has an explicit tracing infrastructure;
- the system has a parser subsystem;
- the system provides support for experiments.

Even before reading the details of each manager, the developer already realizes that the GenESyS core was designed as a coordinating infrastructure and not as a single execution block.

7.4 The class `Model` as the center of the simulation model

The `Model` class deserves special attention because it represents, in the kernel, the simulation model itself. Its importance is already explained in the header documentation: it is described as probably the most important class in the GenESyS kernel. This formulation makes sense. It is where components, *data definitions*, list of future events, controls, responses, persistence, parser, consistency check, entity management and access to the simulation state are articulated.

In practical terms, `Model` is the class in which the model stops being just an idea or diagram and becomes an object that can be manipulated by the software.

7.4.1 Why `Model` is so central

The centrality of `Model` arises from the fact that it articulates several essential mechanisms at the same time:

- insertion and removal of model elements;
- creation and removal of entities;
- sending entities between components;
- model consistency check;
- persistence and loading;
- access to parser, tracing and simulation;
- maintenance of the list of upcoming events.

This means that the `Model` class is not just a component container. It is the organizing center of the model's life within the simulator.

7.4.2 Model as a Combination of Components and Data

The class documentation itself indicates that the model is mainly represented by:

- a collection of model components, appropriately configured and connected;
- a collection of *model data definitions*, which supports the model infrastructure.

This distinction is architecturally fundamental. It anticipates a principle that will return in later chapters: the model is not just made of flow blocks. It also depends on a layer of data and definitions that supports the simulation.

7.4.3 The Future Event List

Among the attributes of `Model`, one deserves special mention: the list of upcoming events. The code's own documentation describes it as one of the most important elements of an event-driven simulation system. This is easily justified. It is from this list that the simulation advances chronologically, always taking the next event to be processed.

For the developer, this is essential. The simulated time does not advance as a simple arbitrary incremental loop. It progresses by orderly firing of scheduled events, and the list of future events is the mechanism that supports this process.

7.5 The class `ModelSimulation`

If `Model` represents the model, `ModelSimulation` represents the execution of that model. This distinction is extremely important. The model and its simulation are related but not identical objects. The first describes the structure of the modeled system; the second controls its execution cycle.

`ModelSimulation`'s documentation is particularly helpful. It describes the current simulation execution layer, based on repetitions, replication length, *warm-up* configuration, and event processing. It also emphasizes that although there are higher abstractions of experimentation in development, `ModelSimulation` remains the execution core used today. This observation is useful for the developer, because it indicates which operational class concentrates the current behavior of the simulation.

7.5.1 Responsibilities of `ModelSimulation`

`ModelSimulation`'s responsibilities can be organized into a few main groups:

- start, pause, step by step and stop the simulation;
- control repetitions;
- maintain the current simulated time;
- manage replication length and period of *warm-up*;
- check termination conditions;
- manage breakpoints;
- trigger reports;
- coordinate event processing.

Note that this class is not a mere operational detail. It concentrates the effective logic of model execution and, therefore, must be studied carefully by any developer who intends to change the central dynamics of the simulator.

7.5.2 The Simulation Cycle

Reading the `ModelSimulation` class allows you to see a clear logical structure of the simulation cycle:

1. simulation initialization;
2. initializing a replication;
3. sequential event processing;
4. checking breakpoints and termination conditions;
5. replication shutdown;
6. simulation termination.

This cycle is very important for understanding the system. It shows that execution is not a simple monolithic "run" call, but a process structured into phases, each with its own responsibilities.

7.5.3 Simulated Time, Replications, and *Warm-up*

One of the merits of the `ModelSimulation` class is that it makes explicit that model execution involves more than simply passing events. There is also the management of:

- current simulated time;
- number of repetitions;
- replication length;
- *warm-up* period;
- base time unit;
- termination condition.

These elements reveal that the class was designed to support simulation-oriented experimentation and analysis, and not just ad hoc execution of a flow of events.

7.6 Events, the Current Event, and Sequential Processing

Discrete event-driven simulation depends on a central idea: the system evolves from event to event. In *GenESyS*, this logic appears very clearly in the relationship between `Model`, `ModelSimulation`, `Event` and the list of future events.

During the simulation, there is always a current event being processed. This event defines the current simulated instant and the context of the ongoing operation. When processed, it can produce effects on entities, components, model data and the system's own future agenda.

This organization has two important implications for the developer:

1. the system state is not updated diffusely, but at discrete processing points;
2. the semantics of the components and most of the model's actions depend on the current event.

This observation connects directly to the parser, entities, components and various expressions linked to the model state, which will be discussed in later chapters.

7.7 The `OnEventManager` class and core observability

If `ModelSimulation` controls execution, `OnEventManager` makes that execution observable through high-level events. This is a very elegant architectural point of GenESyS and deserves special attention.

The `OnEventManager` class allows you to register handlers for different types of occurrences in the model and simulation life cycle. Among them are:

- model check successful;
- model loading and saving;
- replication initiation and termination;
- event processing;
- creating, moving and removing entities;
- start, pause, resume and end of the simulation;
- breakpoint triggering.

This infrastructure shows that GenESyS treats observability as a constitutive part of the kernel, and not as a mere peripheral interface resource.

7.7.1 Why this is architecturally important

`OnEventManager` elegantly separates two things:

- the simulation execution logic;
- the logic for reacting to relevant simulation events.

This separation is valuable because it allows applications, tools, reporting, and debugging mechanisms to plug into kernel behavior without having to mix their logic directly with central processing. It is, in essence, an observation and reaction mechanism based on events in the simulator itself.

7.7.2 ModelEvent e SimulationEvent

The presence of the auxiliary classes `ModelEvent` and `SimulationEvent` reinforces this architecture. They function as context objects for registered handlers, carrying information about the model, the simulated time, the current event, the replication, the paused or stopped state, the created entity and the target component, among other relevant data.

With this, `OnEventManager` not only announces that "something has occurred", but provides enough context for the reaction to that event to be technically useful.

7.8 Breakpoints and Fine-Grained Execution Control

The `ModelSimulation` class also makes explicit the existence of breakpoint lists by time, component and entity. This feature is particularly relevant because it shows that the kernel was designed not only to run models to completion, but to allow controlled observation of the simulation.

From the user's perspective, this appears in the GUI as the ability to pause or advance step by step. From the developer's point of view, this reveals that fine control over execution is not added by the interface in a superficial way. It is provided in the kernel infrastructure itself.

This is an important architectural decision. It means that debugging, inspection, and controlled experimentation do not depend exclusively on the graphical application; they are based on the simulation core.

7.9 An integrated reading of the kernel

The best way to understand the core of *GenESyS* is to integrate the functions of the classes studied so far.

- `Simulator` coordinates access to central subsystems.
- `Model` represents the model, its components, data and future events.
- `ModelSimulation` controls the execution cycle for this model.
- `OnEventManager` makes relevant events observable and responsive.

This integration helps to realize that the kernel is not a set of parallel classes without articulation. It is a coherent infrastructure in which each class occupies a specific role within the same simulation dynamics.

7.10 Final Considerations

This chapter presented the core of the simulator based on its most central classes. The main point to remember is that *GenESyS* organizes its simulation infrastructure clearly: there is a high-level class that coordinates subsystems, a class that represents the model, a class that controls the execution of that model, and an explicit event-driven observability mechanism. This organization is robust enough to support applications, parser, *plugins*, tracing, breakpoints and future extensions.

With this foundation established, the next natural step is to move on to the structure of the model elements themselves. In other words, if this chapter showed how the system

Table 7.1: Roles of the core classes in the GenESyS kernel.

Classe	**Papel principal**
`Simulator`	Highest-level kernel class, coordinating access to central managers
`Model`	Representation of the simulation model, its components, data, entities and future events
`ModelSimulation`	Control of model execution, including repetitions, simulated time, termination conditions and event processing
`OnEventManager`	Registration and triggering of handlers associated with relevant model and simulation events

represents and executes a model, the following chapter will show what types of objects this model is made of: components, *data definitions* and extensibility mechanisms by *plugins*.

Test your understanding of this content by answering the following questions:

1. What is the conceptual difference between `Model` and `ModelSimulation` in the GenESyS kernel?
2. Why is the list of future events such an important structure in event-driven simulation?
3. What role does `OnEventManager` play in system observability?
4. Why is the presence of breakpoints in the kernel a relevant architectural decision and not just an interface detail?

8. Components, *model data* and *plugins* system

8.1 Introduction

After studying the core of the simulator at its most central level, the next step is to understand what concrete objects the model is made of. In other words, if the previous chapter answered the question "how does GenESyS represent and execute a model?", this chapter answers another equally important question: "what are the elements that make up this model within the software?".

In GenESyS, this answer is not reduced to a single class type. The model is built on a fundamental architectural distinction between:

- template components;
- *model data definitions*.

This distinction is especially important because it organizes the entire way the simulator represents the main flow of a system and the data infrastructure that supports it. Furthermore, it is from here that the extensibility mechanism via *plugins* gains clarity. New components and new model data can be introduced to the system without the core having to be rewritten as a monolithic block.

This chapter delves deeper into this architecture. First, it discusses the `ModelDataDefinition` base class. Then, it shows how `ModelComponent` specializes it to represent behavior in the model flow. It then discusses the importance of internal and attached data, the relationship between composition and aggregation and, finally, situates the *plugins* mechanism as a way to expand the system in a consistent manner.

8.2 `ModelDataDefinition` as model base class

The `ModelDataDefinition` class occupies a structural role in GenESyS. Its documentation describes it as the basis for any model data queues, resources, variables, and other elements and also for any model components. This means that it defines a common infrastructure that will be shared both by elements that support the simulation and by elements that directly participate in the model flow.

This design decision is extremely important. Instead of completely separating flow components and supporting data into unrelated hierarchies, GenESyS makes them both share a common foundation. With this, the software gains:

- a uniform identification and naming mechanism;
- common persistence support;
- common verification support;
- common infrastructure for creating internal and attached data;
- standardized tracing mechanisms and simulation controls.

In architectural terms, this makes the model more coherent. Instead of two uncommunicating universes, there is a unified hierarchy in which components and data are different species of model elements.

8.2.1 Core responsibilities of `ModelDataDefinition`

Reading the class shows some particularly relevant responsibilities:

- maintain element identity and name;
- offer persistence infrastructure;
- allow local verification of the element;
- sustain internal and attached data;
- offer integration with parser and simulation controls;
- provide tracing mechanisms;
- defines extension point for creating related data.

This means that `ModelDataDefinition` is not just a convenient abstract superclass. It is a real block of infrastructure on which the rest of the model rests.

8.2.2 Internal Data and Attached Data

One of the most interesting aspects of the `ModelDataDefinition` class is the explicit distinction between *internal data* and *attached data*. This distinction, which has already been important in previous discussions of the book, appears here as part of the system's basic infrastructure.

- **Internal data** corresponds to elements internal to the object, maintained by composition.
- **Attached data** corresponds to referenced data, maintained by aggregation.

This distinction is architecturally very valuable. It allows the software to clearly represent both elements that structurally belong to a component and elements that are only associated with it. In other words, GenESyS does not treat all relationships between model objects in the same way, and this greatly improves the clarity of the system's internal semantics.

8.2.3 Persistence and Verification as Base Responsibilities

Another important aspect is that persistence and verification already appear in the base class. This shows that the system expects model elements to be:

- loaded;
- saved;
- validated;
- reconstituted;
- prepared for simulation.

By concentrating this infrastructure at the base, GenESyS avoids unnecessary repetition in subclasses and imposes consistent discipline on model extensions.

8.3 `ModelComponent` as behavioral specialization

The `ModelComponent` class inherits from `ModelDataDefinition` and adds a decisive dimension to it: behavior guided by entity arrival and event triggering. Its documentation describes it as a block that represents a specific behavior to be simulated and that is activated when an entity arrives at the component.

This formulation is extremely important because it defines the place of the components in the GenESyS architecture. A component is not just a visual object nor just a nominal block of the model. It is a unit of behavior within the flow of the modeled system.

8.3.1 The Idea of Flow Between Components

A model built with `ModelComponent` is, in essence, a set of interconnected components. The global behavior of the model emerges from the path of entities through this flow. This explains

why the class maintains a `ConnectionManager` and provides connection operations between components.

This is one of the big differences between `ModelComponent` and a generic *model data definition*:

- a component directly participates in the flow logic;
- model data can support this flow without itself being part of the main path.

8.3.2 Event dispatch and `_onDispatchEvent`

The `ModelComponent` class makes it clear, through its interface, that each concrete component must implement a specific dispatch behavior. This appears in the protected and pure virtual method `_onDispatchEvent(Entity*, unsigned int)`. It is at this point that the specific semantics of each concrete component comes into existence.

This architectural choice is particularly good because it separates:

- the common infrastructure of the components;
- the particular behavior of each type of component.

With this, GenESyS can have a well-defined base class, but still maintain highly specialized subclasses.

8.3.3 Components as Specialized Model Data

The fact that `ModelComponent` inherits from `ModelDataDefinition` has profound consequences:

- a component also has a name, identity and persistence;
- a component can also have internal and attached data;
- a component participates in the common verification infrastructure;
- a component naturally integrates with the rest of the model's semantics.

This avoids an artificial separation between data structure and behavioral structure. Instead, the system recognizes that a component is also an element of the model, just endowed with a specific behavioral role.

8.4 Main Flow and Supporting Data

One of the most important ideas that the developer needs to consolidate is the difference between:

- the main flow of the model;
- the supporting data that supports this flow.

In practice:

- the main flow is composed of connected components;
- supporting data includes queues, resources, attributes, variables, collectors, counters and other related objects.

This distinction is central because it guides both the modeling and extension of the software. Many design errors occur precisely when trying to treat a support element as if it were a component of the flow, or vice versa.

8.4.1 Why This Distinction Improves the Architecture

By separating main stream and supporting data, GenESyS gains clarity:

- procedural behavior is concentrated in components;
- support objects are represented as model data;
- the relationship between the two can be expressed in an explicit and controlled way;
- the system becomes more extensible.

This also improves the readability of the software. The developer can more easily identify whether a new element they want to implement should be a component, model data, or a combination of the two by composition or aggregation.

8.5 Composition, Aggregation, and the Internal Model Design

The infrastructure of `ModelDataDefinition` and `ModelComponent` reveals that GenESyS takes the distinction between composition and aggregation seriously.

In composition, the internal element structurally belongs to the object that contains it. In practical terms, this means that it is created, maintained, and destroyed as part of the main object's lifecycle. In aggregation, on the contrary, there is only a reference or link to existing data in the model, without absorbing its life cycle.

This distinction is particularly useful when a component:

- internally maintains its own counters or statistics;
- or references external objects such as queues and resources.

Without this separation, the system would run the risk of producing ill-defined links between model elements, making persistence, removal, verification and reuse difficult.

8.5.1 Internal data as a composition relationship

When a component or model data maintains *internal data*, it is explaining a composition relationship. This is often suitable for elements that:

- exists only to support the internal functioning of that object;
- don't make sense as model-independent objects;
- should disappear along with the main object.

8.5.2 Attached data as an aggregation relationship

When an object maintains *attached data*, what exists is an aggregation relationship. The referenced data continues to exist in the model as its own object, and can be shared, referenced by other elements and persisted independently.

This separation between composition and aggregation makes the GenESyS architecture much cleaner and more semantically expressive.

8.6 The *plugins* system

The presence of *plugins* is one of the most important features of GenESyS as extensible software. The `Simulator` class itself already makes this evident when exposing a `PluginManager`. In the current repository, the plugin implementations are built as static libraries and linked into the simulator, while the connector and manager layers organize metadata, discovery and instantiation. This shows that the *plugins* mechanism is neither improvised nor peripheral; it is a structural part of the system, but not a set of independent runtime packages.

For the developer, the consequence is clear: expanding GenESyS does not necessarily mean directly modifying the kernel. In many cases, the most appropriate strategy is to introduce new capabilities through *plugins*. This is especially true for:

- new components;
- new *data definitions*;
- new language extensions;
- new specialized behaviors.

8.6.1 Why *plugins* are important

Without *plugins*, the system's growth would depend on increasingly profound changes to the core. This would tend to make the software more rigid and more difficult to maintain. With *plugins*, the project gains:

- modularity;
- extensibility;
- domain specialization;
- greater separation between stable infrastructure and additional functionality.

This is an especially appropriate architectural decision for a simulator that aims to be generic and expandable. At the current stage, that extensibility is expressed at build and link time, not as a separate deployable plugin runtime. Any future ABI boundary belongs to a later architectural decision, not to the present contract described here.

8.6.2 Components and Data as Natural Extensions

From the model's point of view, many *plugins* precisely materialize as new components and new data. This means that the architecture discussed so far `ModelDataDefinition`, `ModelComponent`, internal data, attached data is not just a matter of internal elegance. It is the foundation on which the system's extensibility rests.

In other words, understanding the architecture of the model elements is also understanding the architecture of *plugins*.

8.7 How to Think About a New Extension

A developer looking to extend GenESyS should start with a very simple question:

> What exactly am I trying to introduce into the system?

From this question, it is usually possible to distinguish some cases:

- a new behavior in the model flow;
- new data supporting the model;
- a new combination of behavior and internal data;
- a new language or parser needs;
- a new application or tool.

If the need is for new behavior in the flow, the extension will probably be in the form of new `ModelComponent`. If it is a new backing object, it will probably take the form of new `ModelDataDefinition`. If it involves both, it will be necessary to think carefully about the composition, aggregation and internal relationships of the new element.

8.8 Misreading This Chapter Avoids

This chapter seeks to avoid a very common misreading: imagining that the GenESyS model is composed only of visual boxes in the GUI. In reality, the model is a much richer structure, supported by a hierarchy of classes and a clear architectural distinction between behavior and data.

It also seeks to avoid another error: imagining that *plugins* are just dynamic loading details or independent runtime packages. In GenESyS, they are part of the system's growth logic and, in the current implementation, are aggregated through static libraries and connector-managed metadata. Extensibility is not an appendix; it is an architectural principle.

Table 8.1: Central Distinctions for Understanding the Model Structure in GenESyS.

Distinction	Architectural Meaning
`ModelComponent` vs. `ModelDataDefinition`	Difference between in-stream behavior and model data/infrastructure
Main stream vs. supporting data	Difference between procedural path and elements that support execution
Internal data vs. data	Difference between composition and aggregation within the model
Kernel vs. *plugins*	Difference between core infrastructure and specialized extensions

8.9 Final Considerations

This chapter showed that the GenESyS model is structured based on a coherent and extensible architecture. The `ModelDataDefinition` class provides the base infrastructure for model elements. The `ModelComponent` class specializes this base to represent behavior in the flow. The distinction between main flow and supporting data, as well as between composition and aggregation, clearly organizes the internal semantics of the system. Finally, the *plugins* mechanism appears as a natural consequence of this architecture, allowing the simulator to be expanded within the current static-linking and metadata-based infrastructure without losing coherence.

With this, the reader already has the necessary basis to understand how new elements can be introduced into the system. The next natural step is to study the subsystem that connects the language, expressions and semantics of the model: the GenESyS parser and its relationship with the simulator's internal language.

Test your understanding of this content by answering the following questions:

1. What is the architectural advantage of making `ModelComponent` inherit from `ModelDataDefiniti`
2. How does the distinction between main stream and supporting data help you understand the structure of the model?
3. What is the difference between *internal data* and *attached data* in the internal design of GenESyS?
4. Why should the *plugins* system be understood as a structural part of the architecture and not just as an optional extension mechanism?

9. Parser, Expressions, and Internal Language

9.1 Introduction

Throughout the first half of this book, the *GenESyS* language appeared to the user mainly through the *Simulang* tab. At that stage, the central interest was in realizing that the graphic model also has a textual representation. For the developer, however, this observation is just the starting point. The real problem now is different: how the system interprets expressions, how the language is linked to the model state and how new syntactic and semantic capabilities can be introduced into the simulator.

This chapter deals exactly with this subsystem. Its focus is not just on the generic notion of "parser", but on the concrete way *GenESyS* organizes the evaluation of expressions, the handling of model symbols, the integration with stochastic functions, the Bison/Flex grammar, and the extension points that allow the system to grow.

The importance of this topic is great. In a simulator like *GenESyS*, language is not ornament. Many model decisions involve expressions: parameters, formulas, attribute references, statistical queries, simulation controls and probabilistic functions. The parser, therefore, is not just a textual convenience; it is part of the operational semantics of the simulator.

9.2 The Role of the Parser in GenESyS

The *GenESyS* parser should be understood as the infrastructure that makes it possible to interpret expressions and functions in model context. This means that it doesn't just operate on abstract text, but on text that needs to be related to concrete elements of the simulator:

- attributes of the current entity;
- simulation controls and responses;
- accountants and statistical collectors;
- variables, formulas, queues, resources and other model objects;
- mathematical and probabilistic functions;
- information about the current state of the simulation.

In other words, the parser works as a bridge between language and the internal state of the system. This is why it is so important in the GenESyS architecture.

9.2.1 Parser as a Semantic Subsystem

It is useful to avoid a limited reading of the parser as a simple parser. In *GenESyS*, it does more than recognize a well-formed expression. It also needs:

- resolves symbol names;
- identify the nature of these symbols;
- get values associated with the current state of the model;
- integrate stochastic functions supported by a sampler;
- propagate useful error messages.

This means that the correct reading of the parser in GenESyS is semantic and not just syntactic.

9.3 The `Parser_if` interface

The `Parser_if` interface offers a very clear summary of the role of the parser in the project. It defines a minimal contract for evaluating stochastic expressions and functions, making some central responsibilities explicit:

- evaluate expressions and return a numeric value;
- offer a variant that reports success and an error message without throwing an exception;
- make the last error message available;
- associate the parser with an instance of `Sampler_if`;
- expose the internal parser driver for advanced scenarios.

This interface is important because it shows that the parser is seen, architecturally, as a simulator service and not as an accidental implementation detail.

9.3.1 Expression Evaluation

The most obvious point of the interface is the expression evaluation method. The existence of two main forms of `parse` is particularly revealing:

- a way that can throw exceptions;
- a way that avoids exceptions and explicitly reports success and error messages.

This decision improves the integration of the parser with different layers of the system. In some contexts, the developer may prefer exception handling. In others, especially in validation and user interface, it is more convenient to receive the success state explicitly.

9.3.2 Integration with `Sampler_if`

The interface also makes it clear that the parser is not just a parser of deterministic expressions. It integrates stochastic functions supported by `Sampler_if`. This is important because it reinforces the specific nature of the GenESyS language: it is not just a mini-arithmetic language, but a simulation-oriented language.

In practical terms, this means that probabilistic functions of the language depend on the presence of a sampler associated with the parser. As a result, the semantics of expressions naturally incorporate random generation mechanisms.

9.4 The `Model` class as parser user

The role of the parser within the kernel is especially clear in the `Model` class. It exposes methods like `parseExpression` and `checkExpression`, which shows that the model uses the parser directly to:

- evaluate expressions defined in the model context;
- validate expressions entered by the user;
- check references to *data definitions* in the text of expressions.

This detail is architecturally very important. It means that the parser is not isolated in an external layer; he participates in the model's life.

9.4.1 Why does `Model` invoke the parser

The presence of these methods in `Model` reveals an important principle: expressions are part of the model's semantics, and not just the interface. A model component or data that stores an expression needs to be able to validate it and, in many cases, evaluate it within the current context of the simulation.

This architectural choice is very good because it keeps the language connected to the object where it really belongs: the model.

9.4.2 Expressions and Consistency Checking

It's also important to note that the `Model` class offers `checkExpression`. This shows that the parser participates not only in the execution, but also in the model checking phase. In other words, the validity of an expression is treated as part of the integrity of the model, and not as a problem to be discovered only during an execution that has already started.

This decision greatly improves the robustness of the system and helps explain why model checking is such an important step in the use and development of GenESyS.

9.5 The GenESyS Bison/Flex Grammar

The concrete implementation of the parser in `source/parser/parserBisonFlex` shows a well-defined implementation of this subsystem. The `bisonparser.yy` file defines a C++ grammar based on `lalr1.cc`, with its own parser class, typed tokens, localizations, support code, and explicit integration with `genesyspp_driver`. This shows that the parser was designed as an explicitly documented component of the simulator infrastructure, and not as an improvised textual routine.

Grammar covers, among other elements:

- numbers;
- relational and logical operators;
- trigonometric and mathematical functions;
- probabilistic functions;
- functions associated with the kernel;
- template symbols;
- extensions introduced by *plugins*.

This scope is enough to show that the parser is one of the subsystems with the highest semantic density in the project.

9.5.1 A Language Truly Integrated with the Model

Reading the grammar shows something particularly relevant: many tokens and productions do not just refer to abstract syntactic constructions, but to real objects in the model. There are rules for attributes, simulation responses, simulation controls, and elements such as variables, formulas, queues, and resources. Functions associated with statistics, counters and simulation status also appear.

This reinforces an essential point:

> The GenESyS language is not external to the simulator; it is a textual projection of the semantics of the model and simulation.

9.5.2 Probabilistic Functions and Sampling

The grammar makes explicit the presence of probabilistic functions such as exponential, normal, uniform, Weibull, lognormal, gamma, Erlang, triangular and beta, among others. In all of them, the operational semantics depend on the `Sampler_if` associated with the parser.

This is very important for the developer, because it shows that:

- the simulator language directly incorporates stochastic sampling;
- the parser implementation needs to maintain a consistent contract with the statistical subsystem;
- error handling in these functions is not only syntactic but also operational.

9.6 Model Symbols and Dependence on the Current State

One of the most interesting features of the GenESyS parser is its explicit dependence on the current state of the simulation. The grammar shows, for example, that access to attributes depends on the current entity associated with the current event. Likewise, functions such as `TNOW`, `TFIN`, number of repetitions and identifiers of the current entity depend on the state maintained in `ModelSimulation` and the current event.

This feature is architecturally important because it reveals that the interpretation of expressions in GenESyS is contextual. The same expression can have different meanings depending on:

- the loaded model;
- the current entity;
- the event being processed;
- the simulated instant;
- the model data available in that context.

9.6.1 Parser como consumidor do estado do kernel

This means that the parser directly consumes information from the kernel. It queries:

- the model;
- the simulation;
- the current event;
- or *data manager*;
- collectors and counters;
- controls and responses.

This dependence is not a defect. Rather, it is part of the nature of the problem. A parser for a simulation language needs to know the universe of symbols and states of the simulator.

9.7 Extension points by *plugins*

The grammar of *GenESyS* very clearly reveals the existence of regions designed for extension. The blocks marked by structured comments for *plugins* inclusions, tokens, types, and productions show that the system is designed for growth.

This aspect is particularly important for the developer. It shows that expanding the language does not necessarily depend on completely rewriting the base grammar. There are architecturally planned points for new elements to be introduced.

9.7.1 Syntactic and Semantic Extension

When a *plugin* extends the language, it doesn't just introduce new tokens. In general, it also introduces:

- new symbols;
- new functions;
- new interpretation rules;
- new relationships between text and model data.

This means that the parser extension is simultaneously syntactic and semantic. This is precisely why the `ParserChangesInformation` and `ParserManager` infrastructure exists in the project.

9.8 The class `ParserManager`

The `ParserManager` class makes explicit GenESyS's architectural ambition to support language evolution. Its interface provides:

- generation of a new parser based on changes;
- connecting a new parser to the system.

Although the header itself indicates that important parts of this flow are still under development, the existence of this class is already architecturally relevant. It shows that GenESyS does not treat grammar as an entirely static artifact. There is, in the system design, space for explicit management of the evolution of the parser.

9.8.1 What This Means for the Developer

For the developer, this means that working with language in GenESyS is not limited to modifying a Bison or Flex file. The project foresees, at an architectural level, a subsystem responsible for:

- describe changes;
- generate parser artifacts;
- connect new parsers to the simulator.

Even though this infrastructure is still under development, it already guides a sensible way of thinking about the evolution of language.

9.9 Erros, mensagens e robustez

Another important aspect of parser implementation is error handling. The grammar shows explicit concern for useful messages, including in the case of illegal tokens, missing literals, and failures in probabilistic functions. This is important because simulation language, when misinterpreted or misdiagnosed, quickly becomes a source of frustration for user and developer.

The robustness of the parser therefore depends on three factors:

- correct syntactic recognition;
- correct semantic integration with the model;
- producing sufficiently informative error messages.

This triad is especially important in a system where expressions can appear at many points in the model.

9.10 How to Study and Modify the Parser Productively

The most productive way to study the GenESyS parser is in layers.

First, understand the `Parser_if` interface and the way `Model` invokes the parser. Then, study the Bison/Flex grammar and the way it accesses model symbols. Then identify the extension points introduced by *plugins*. Only then does it make sense to propose more significant changes, especially when they involve new functions, new symbols or changes to grammar.

It is also important to remember that changes to the parser are not just textual changes. They can affect:

- model checking;
- component semantics and *data definitions*;
- integration with sampling;
- error messages;
- compatibility with *plugins*.

9.11 Final Considerations

This chapter has shown that the *GenESyS* parser is much more than an auxiliary text layer. It is an integral part of the simulator semantics. The `Parser_if` interface, the direct integration with `Model`, the Bison/Flex grammar, the use of model and simulation symbols, the

Table 9.1: Reading Layers of the GenESyS Parser.

Camada	**Pergunta principal**
Interface	What service does the parser provide to the rest of the system?
Model Integration	How does `Model` and Simulation State use this service?
Concrete grammar	How is language recognized and evaluated?
Extensibility	How do new symbols and functions enter the system?
Robustness	How are errors detected and reported?

dependence on a sampler, and the explicit extension points by *plugins* reveal an architecturally dense and strategically important subsystem for the project.

With this, the reader already has the main elements to understand how *GenESyS* links language, model and execution. The next step is to broaden the focus again, leaving the parser and returning to the project's broader ecosystem: applications, tools, C++ examples, tests and software evolution strategies. This is exactly what will be covered in the next chapter, which closes this version of the developer manual.

Test your understanding of this content by answering the following questions:

1. Why should the *GenESyS* parser be understood as a semantic subsystem and not just a syntactic one?
2. How important is it for the `Model` class to expose methods like `parseExpression` and `checkExpression`?
3. How does the *GenESyS* grammar show dependence on the current state of the model and simulation?
4. Why are extension points by *plugins* essential to the evolution of the simulator language?

10. Applications, Tools, C++ Examples, Tests, and Project Evolution

10.1 Introduction

This chapter wraps up the main structure of this developer manual by broadening the focus again. In the previous chapters, the attention fell on the central architecture of the system: organization of the source code, kernel, structure of model elements, *plugins*, and parser. Now the goal is to examine how this infrastructure projects into concrete applications, helper tools, programmatic examples in C++, and testing mechanisms. In other words, this chapter is about the development ecosystem that forms around the *GenESyS* core.

The importance of this closure is great. An extensible simulator doesn't just live off its core classes. It also depends on applications that expose it to the user, on tools that support its use and maintenance, on examples that show how the library can be used programmatically and on tests that support the safe evolution of the system. Without these elements, the core of the software could exist, but the project as a development platform would be impoverished.

Therefore, this chapter should be read as a final synthesis of the second half of the book. It shows how GenESyS transitions:

- from the kernel to applications;

- of the architecture for programmatic use;
- functional extension for technical maintenance;
- of stable infrastructure for the future evolution of the project.

10.2 Applications as Projections of the Kernel

The `source/applications` folder is the place where GenESyS stops appearing just as infrastructure and starts to take on concrete forms of interaction. This observation is essential for the developer. It shows that the project was not designed to exist just as an abstract library, but to support real applications at its core.

Architecturally, this is very valuable. By separating the kernel from the applications, the system gains:

- modularity;
- core reuse on different interfaces;
- lower coupling between simulation logic and form of interaction;
- greater capacity for evolution.

In practice, this separation allows the GUI, Shell application, model-specific applications, and other future layers to exist as distinct projections of the same set of core services.

10.2.1 The GUI as an Application on the Core

In the first half of this book, the GUI was treated as the main way of using the simulator. Now, it starts to be seen in another way: as an application built on the GenESyS infrastructure. This shift in perspective is especially important for the developer.

The GUI is not the core. It is an application that:

- consumes the model;
- consumes the semantics of components and data;
- connects to simulation engines;
- exposes these capabilities to the user through a visual interface.

This separation is architecturally correct and explains why the system can, in principle, support other applications in addition to the graphical interface.

10.2.2 The Shell and Model-Specific Applications as Alternative Forms of Use

The current build tree of the project shows that the command-line applications are handled in a particularly structured way. The `CMakeLists.txt` of `source/applications/shell` defines the `genesys_shell` target, while `source/applications/modelSpecific` defines `genesys_modelspecific_app` and lets the build select a concrete model-specific example file to compose the executable.

This is extremely important for the developer. It shows that the command-line path is not just an old auxiliary tool or a marginal way of use. It is an effective application of GenESyS and, more than that, a valuable environment for:

- study the programmatic use of the simulator;
- build compact examples;
- develop lightweight applications based on the library;
- check behaviors without GUI mediation.

The *Worker* service remains outside this application path as a local or private component only; this chapter does not provide public deployment instructions or imply that public exposure is approved.

10.3 The examples in `C++` as developer material

In the first half of the book, the examples in `C++` of the command-line applications were deliberately left out, because the user manual should not start with code. Now, however, they begin to occupy a central place. This is because these examples show one of the most important capabilities of GenESyS as software: its use as a simulation library.

The model-specific application's `CMakeLists.txt` explicitly shows the existence of build-selectable examples, including *smart* examples and more elaborate teaching examples, which confirms the role of these files as study material and architectural demonstration.

10.3.1 Why These Examples Are So Valuable

The examples in `C++` are valuable because they allow you to observe the simulator from another angle. In the GUI, the model appears as a visual and operational artifact. In the programmatic examples, it appears as an object explicitly constructed by the developer. This helps reveal:

- how *Simulator* is instantiated;
- how the model is built by code;
- how components and *data definitions* are created and related;
- how execution is triggered;
- how the simulator library can be reused in new applications.

10.3.2 The bridge between examples in `C++` and saved models

One of the most didactically rich features of the project is the possibility of relating:

- examples built programmatically in `C++`;
- models saved in the `models/` folder.

This bridge is extremely useful for the developer because it shows the same system from two perspectives:

- as library object;
- as a modeling and usage artifact.

This correspondence helps to consolidate the understanding of GenESyS as a platform, and not just as a ready-made application.

10.4 Tools as a Support Layer

The `source/tools` folder represents an important region of the project, although it is often less discussed than the kernel, parser, and *plugins*. Its general function is to support system use, development, integration, or maintenance.

From a developer's point of view, tools are relevant for three reasons:

- help make the simulator ecosystem more productive;
- offer points of support for observation or automation;
- can serve as a basis for new integrations.

In architectural terms, the existence of `tools` shows that the project is not limited to the "kernel versus application" dichotomy. There is also space for intermediate instruments, utilities and support mechanisms.

10.4.1 When to study `tools`

Tools should not always be the first reading priority. In general, it makes more sense to study them after the developer already understands:

- the simulator core;
- the *plugins* system;
- the parser;
- at least one concrete application, such as the GUI or the Shell.

After that, reading the tools tends to be much more productive, because the developer can better see their role in the project ecosystem.

10.5 The Importance of Tests

In a project of the size and ambition of *GenESyS*, the existence of tests is an essential part of the software's evolutionary architecture. This not only means that the project "has tests", but that the build system itself already treats them as a structured part of development.

`CMakeLists.txt` of `source/tests` explicitly shows the division between unit tests and *smoke tests*, with an aggregated target `genesys_tests` and the possibility of enabling or disabling the construction of smoke tests by build option

This organization is particularly valuable because it gives the developer a way to think

about validation in layers:

- unit tests for local behavior and minor invariants;
- *smoke tests* for broader system health.

10.5.1 Why Tests Are Part of the Architecture, Not Just the Routine

It is common to think of testing only as an activity external to the main project. In GenESyS, however, they must be read as part of the system's evolutionary infrastructure. This is particularly important because the software has:

- relatively dense kernel;
- extension mechanisms;
- multiple applications;
- integration between parser, model, events and observability.

In a context like this, changing code without testing support tends to be risky. Therefore, the developer must see the `source/tests` folder as a direct ally in project maintenance.

10.5.2 How the Developer Should Use the Tests

A disciplined development practice in GenESyS must consider testing in at least three moments:

1. before the change, to understand the current basis of behavior;
2. during the change, to avoid local regressions;
3. after the change, to validate integration and general sanity.

This discipline is especially important when the change affects the kernel, parser, *plugins* or applications.

10.6 Build, Applications, and Behavior Selection

Even though the build is not the guiding axis of the book, it continues to offer the developer an important set of signals about the project organization. The Shell and model-specific build files, for example, show that the system was designed to allow configurable selection of the behavior of the command-line executable, either by a fixed Shell entry point or by concrete examples under `source/applications/modelSpecific`.

This reveals something important about the project:

> GenESyS is not just a program; it is a platform upon which application behaviors can be chosen and assembled.

This observation is of great interest to the developer because it reinforces the modularity of the system and helps to think about future applications about the core.

10.7 Project Evolution and Contribution Paths

The combined reading of kernel, parser, *plugins*, applications, tools and tests shows that GenESyS is a project in continuous evolution. This evolution can occur on several fronts:

- kernel improvements;
- expansion of the set of components and *data definitions*;
- evolution of the language and parser;
- strengthening the GUI and other applications;
- expansion of auxiliary tools;
- consolidation of the test base.

From the developer's point of view, the most useful question is no longer:

> Where can I move?

and becomes:

> At which layer of the system does my change make the most sense architecturally?

This change in perspective is what distinguishes an opportunistic modification from a technically grounded contribution.

10.7.1 How to Choose an Entry Point for Contribution

The choice of entry point depends largely on the type of contribution desired.

- If the intention is to change the central dynamics of the simulation, the study must begin in the kernel.
- If the intention is to expand language and expressions, the study must begin with the parser.
- If the intention is to add modeling elements, reading should focus on *plugins*, components and *model data*.
- If the intention is to improve the user experience, the GUI and applications become central.
- If the intention is to increase robustness, testing and tracing become a priority.

This way of thinking greatly improves the quality of the contribution to the project.

10.7.2 The Role of Examples and Tests in Evolution

Examples and tests play complementary roles in software evolution.

- Examples show como usar the system and help you explore new capabilities.
- Tests help verify se o sistema continua correto when being modified.

A GenESyS developer needs to value both. Without examples, the system loses its ability to demonstrate and learn. Without testing, you lose evolutionary security.

10.8 An Integrated Final View of GenESyS

Coming to the end of this version of the developer manual, it is useful to return to the integrated view of the project.

GenESyS can be understood as:

- a simulation kernel;
- a hierarchy of objects representing model, components and data;
- a language system and parser;
- an extensibility infrastructure by *plugins*;
- a set of applications built on top of the kernel;
- an ecosystem of examples, tools and tests that supports use and evolution.

This integrated view is important because it avoids too partial readings. GenESyS is not just GUI, not just parser, not just *plugin framework*, not just simulation library. It is the articulation of all these dimensions.

Table 10.1: Main Dimensions of GenESyS as a Software Platform.

Dimension	**Role in the Project**
Kernel	Central Simulation Infrastructure
Parser	Language, expressions and textual semantics of the model
Plugins	Simulator extensibility and expansion of modeling elements
Applications	Concrete forms of use, such as GUI and Shell application
Tools	Support for use, integration and maintenance
Tests	Evolutionary security, validation and regression prevention
Examples	Material for demonstration, study and programmatic use of the library

10.9 Final Considerations

This chapter has closed the main structure of the developer's manual by showing that *GenESyS* should not only be read for its central classes, but also for its ecosystem of applications, tools, examples and tests. The GUI now appears as an application on top of the core. The Shell application and the model-specific examples in `C++` reveal the use of the simulator as a library. Tools expand the development environment. The test tree provides the basis for safe evolution. And the general organization of the project shows that GenESyS should be thought of as a software platform, and not just as a ready-made simulator.

With this, the reader now has a sufficiently broad and technically grounded view to use, study and expand GenESyS in a careful way. From here, further development can follow different paths: review and improvement of the GUI, more detailed study of components and *plugins*, evolution of the parser, strengthening of the testing infrastructure, development of new applications or creation of new modeling domains on the same core. In all these cases, the logic remains the same: understand the correct layer, respect the system architecture and evolve the simulator based on its own structural principles.

Test your understanding of this content by answering the following questions:

1. Why should the GUI be understood, from the developer's point of view, as an application on the core and not as the entire system?
2. What is the architectural value of the examples in `C++` of the Shell and model-specific applications?
3. Why should the project's test base be treated as part of the software evolution infrastructure?
4. How does the distinction between kernel, parser, *plugins*, applications, tools, examples and tests help the developer to better choose the entry point for a contribution?

Bibliography